

[图编译器串讲] fusion & sharding & tiling

1. 背景

fusion, sharding, tiling是图编译器中对推理性能影响最大的模块, 解决的问题:

1. 配合硬件multi-core和multi-level buffer, 进行数据切分, 最大能力实现并行计算和IO隐藏

从算子角度看, 分为两部分:

1. 算子自身在multi-core和multi-level buffer上计算行为, 即sharding和tiling
2. 算子与算子之间的数据通信, 即fusion

需要决策的问题:

1. 分配几个npu core用于单算子计算
2. 算子负载需要在哪些层buffer进行tile, 每一层的tile-size
3. producer算子和consumer算子在哪一层buffer做数据通信

上述问题的决策空间非常大, 问题的求解属于NP-hard. 所以编译器的任务是在可接受编译时间内, 寻找一个次优解.

编译器需要解决的问题:

1. 如何构造解空间, 不漏解; 同时要求IR能够表达各种可行解, 比如flash-attention方案
2. 快速的在解空间中搜索出最优解或次优解

2. 调研

参见 [📖 holo-graph-compiler 3.2节](#)

主流编译器的实现路线可以归结如下:

1. 启发式: 用一组经验公式/阈值, 直接决定切谁、切几层、每维tile多大、要不要fuse
2. AutoSchedule: 构建解空间, 使用智能算法或机器学习算法搜索可行解, 上机评测, 更新数据库/cost-model; 推理时查询数据库或走cost-model评估获取融合子图的最佳切分方案
3. Polyhedral model: 通过将嵌套循环的迭代空间抽象为高维整数凸多面体, 利用仿射变换将其切分为满足数据依赖约束且最大化缓存局部性与并行度的tile块.

参考主流编译器和过往开发经验, 根据我们自研NPU架构, holo-graph-compiler做如下考量:

调度模式/编程模型

1. 整体切分方案基于SPMD模式, 在此基础上, 兼容Dataflow

性能与实施

1. 对于决定性能的关键决策, 优先保留其完整的决策空间; 其它决策点可合理剪枝
2. 性能的关键是减少或隐藏IO, 特别是DDR的访存; 所有切分决策的优先考量是减少或隐藏DDR的访存 latency
3. 最大限度的根据硬件特性, 提取经验公式, 保证性能的同时, 极大压缩解空间. 如根据mma/vector单指令数据处理规模(32x32), 作为tiling时的align size
4. 优先保证功能兜底, 实施复杂度适中, 可调试性好(迭代快)
5. 多级pass, 每级pass完成一个或多个决策点的确认, 最终完成所有决策
6. 将决策点分成两类, 有经验公式的一类基于启发式(求解速度快, 性能有保证), 复杂类决策基于 AutoSchedule

3. 下降流程

3.1 onnx-to-mlir pass

代码块

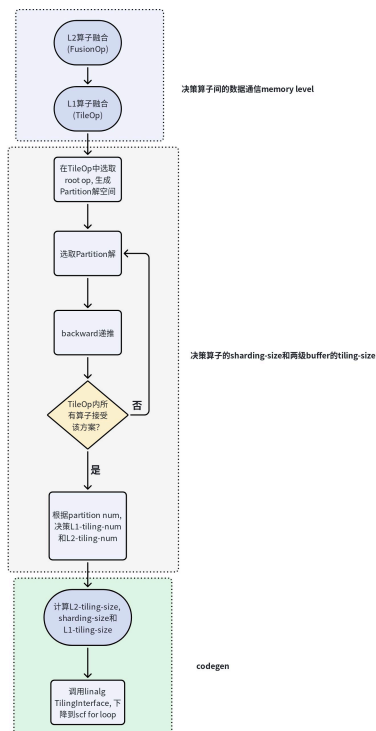
```
1  #loc = loc(unknown)
2  module attributes {top.core_num = 64 : i64, top.version = "1.0.0"} {
3    func.func @AttentionGraphFixed(%arg0: tensor<1x2048x512xf32> loc(unknown)) -
      > tensor<1x2048x1024xf32> {
4      %0 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<1x2048x2048xf32>}> : () -> tensor<1x2048x2048xf32> loc(#loc1)
5      %1 = "top.constant"() <{value = dense<3.125000e-02> : tensor<f32>}> : () -
        > tensor<1x1x1xf32> loc(#loc2)
6      %2 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc3)
7      %3 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc4)
8      %4 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc5)
9      %5 = "top.matmul"(%arg0, %4) <{transB = false}> : (tensor<1x2048x512xf32>,
        tensor<1x512x1024xf32>) -> tensor<1x2048x1024xf32> loc(#loc6)
10     %6 = "top.matmul"(%arg0, %3) <{transB = false}> : (tensor<1x2048x512xf32>,
        tensor<1x512x1024xf32>) -> tensor<1x2048x1024xf32> loc(#loc7)
```

```

11     %7 = "top.matmul"(%arg0, %2) <{transB = false}> : (tensor<1x2048x512xf32>,
    tensor<1x512x1024xf32>) -> tensor<1x2048x1024xf32> loc(#loc8)
12     %8 = "top.transpose"(%6) <{perm = [0, 2, 1]}> : (tensor<1x2048x1024xf32> -
    > tensor<1x1024x2048xf32> loc(#loc9)
13     %9 = "top.matmul"(%5, %8) <{transB = false}> : (tensor<1x2048x1024xf32>,
    tensor<1x1024x2048xf32>) -> tensor<1x2048x2048xf32> loc(#loc10)
14     %10 = "top.mul"(%9, %1) : (tensor<1x2048x2048xf32>, tensor<1x1x1xf32>) ->
    tensor<1x2048x2048xf32> loc(#loc11)
15     %11 = "top.add"(%10, %0) : (tensor<1x2048x2048xf32>,
    tensor<1x2048x2048xf32>) -> tensor<1x2048x2048xf32> loc(#loc12)
16     %12 = "top.softmax"(%11) <{axis = 2 : i64}> : (tensor<1x2048x2048xf32>) ->
    tensor<1x2048x2048xf32> loc(#loc13)
17     %13 = "top.matmul"(%12, %7) <{transB = false}> : (tensor<1x2048x2048xf32>,
    tensor<1x2048x1024xf32>) -> tensor<1x2048x1024xf32> loc(#loc14)
18     return %13 : tensor<1x2048x1024xf32> loc(#loc15)
19 } loc(#loc)
20 } loc(#loc)

```

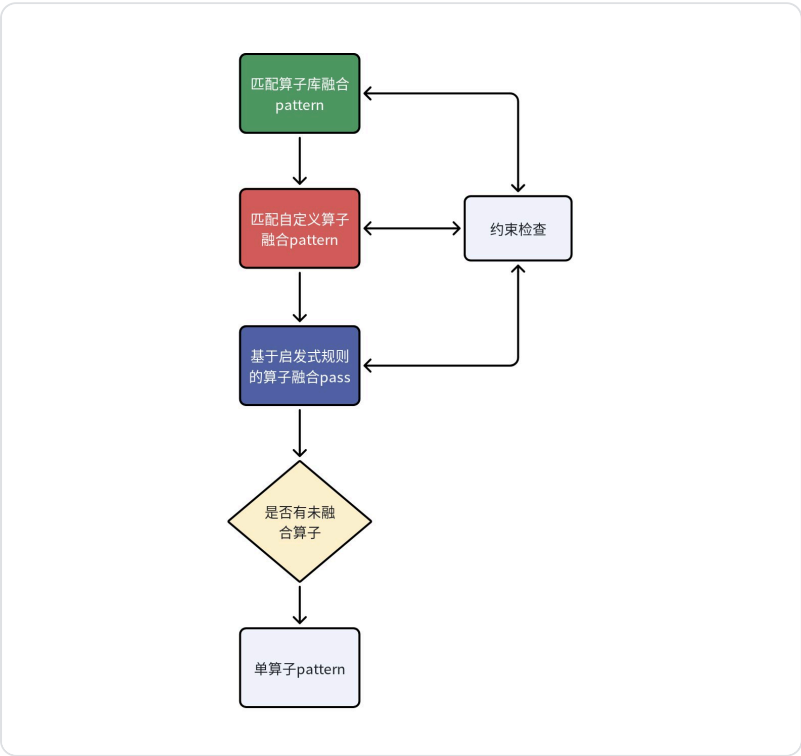
基于以上考量, fusion & sharding & tiling的下降流程如下



3.2 l2-op-fusion pass

一组算子group, 以FusionOp表示, 数据从DDR进, DDR出. 支持三级融合策略

待开发项	优先级
算子库kernel fusion pattern	高
自定义kernel fusion pattern	中
启发式kernel fusion pattern	低



考量:

- 1. 与算子库的kernel概念对齐, 适配call oplib路线
- 2. 决策哪些算子在L2融合

代码块

```
1  #loc = loc(unknown)
2  module attributes {top.core_num = 64 : i64, top.version = "1.0.0"} {
3    func.func @AttentionGraphFixed(%arg0: tensor<1x2048x512xf32> loc(unknown)) -
      > tensor<1x2048x1024xf32> {
4      %0 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<1x2048x2048xf32>}> : () -> tensor<1x2048x2048xf32> loc(#loc1)
```

```

5      %1 = "top.constant"() <{value = dense<3.125000e-02> : tensor<f32>}> : () -
> tensor<1x1x1xf32> loc(#loc2)
6      %2 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc3)
7      %3 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc4)
8      %4 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc5)
9      %5 = top.fusion {
10         %6 = "top.fusionInput"(%arg0) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32> loc(#loc6)
11         %7 = "top.fusionInput"(%0) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32> loc(#loc6)
12         %8 = "top.fusionInput"(%1) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32>
loc(#loc6)
13         %9 = "top.fusionInput"(%2) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32> loc(#loc6)
14         %10 = "top.fusionInput"(%3) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32> loc(#loc6)
15         %11 = "top.fusionInput"(%4) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32> loc(#loc6)
16         %12 = "top.matmul"(%6, %11) <{transB = false}> :
(tensor<1x2048x512xf32>, tensor<1x512x1024xf32>) -> tensor<1x2048x1024xf32>
loc(#loc7)
17         %13 = "top.matmul"(%6, %10) <{transB = false}> :
(tensor<1x2048x512xf32>, tensor<1x512x1024xf32>) -> tensor<1x2048x1024xf32>
loc(#loc8)
18         %14 = "top.transpose"(%13) <{perm = [0, 2, 1]}> :
(tensor<1x2048x1024xf32>) -> tensor<1x1024x2048xf32> loc(#loc9)
19         %15 = "top.matmul"(%12, %14) <{transB = false}> :
(tensor<1x2048x1024xf32>, tensor<1x1024x2048xf32>) -> tensor<1x2048x2048xf32>
loc(#loc6)
20         %16 = "top.mul"(%15, %8) : (tensor<1x2048x2048xf32>, tensor<1x1x1xf32>) -
> tensor<1x2048x2048xf32> loc(#loc10)
21         %17 = "top.add"(%16, %7) : (tensor<1x2048x2048xf32>,
tensor<1x2048x2048xf32>) -> tensor<1x2048x2048xf32> loc(#loc11)
22         %18 = "top.softmax"(%17) <{axis = 2 : i64}> : (tensor<1x2048x2048xf32>) -
> tensor<1x2048x2048xf32> loc(#loc12)
23         %19 = "top.matmul"(%6, %9) <{transB = false}> : (tensor<1x2048x512xf32>,
tensor<1x512x1024xf32>) -> tensor<1x2048x1024xf32> loc(#loc13)
24         %20 = "top.matmul"(%18, %19) <{transB = false}> :
(tensor<1x2048x2048xf32>, tensor<1x2048x1024xf32>) -> tensor<1x2048x1024xf32>
loc(#loc14)
25         %21 = "top.fusionOutput"(%20) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32> loc(#loc6)
26         top.return %21 : tensor<1x2048x1024xf32> loc(#loc6)
27     } : tensor<1x2048x1024xf32> loc(#loc6)

```

```

28     return %5 : tensor<1x2048x1024xf32> loc(#loc15)
29   } loc(#loc)
30 } loc(#loc)
31

```

3.3 l1-op-fusion pass

一组算子group, 以TileOp表示, 包含在FusionOp内, 数据从L2进, L2出(只支持单输出). 支持两级融合策略

待开发项	优先级
自定义fusion pattern	高
启发式fusion pattern	低

考量:

1. 决策哪些算子在L1融合
2. memory和调度层次更清晰; 每个TileOp都可以单独分配tile-mask
 - a. 更好的负载均衡
 - b. 方便实施dataflow

代码块

```

1  #loc = loc(unknown)
2  module attributes {top.core_num = 64 : i64, top.version = "1.0.0"} {
3    func.func @AttentionGraphFixed(%arg0: tensor<1x2048x512xf32> loc(unknown)) -
      > tensor<1x2048x1024xf32> {
4      %0 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<1x2048x2048xf32>}> : () -> tensor<1x2048x2048xf32> loc(#loc1)
5      %1 = "top.constant"() <{value = dense<3.125000e-02> : tensor<f32>}> : () -
        > tensor<1x1x1xf32> loc(#loc2)
6      %2 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc3)
7      %3 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc4)
8      %4 = "top.constant"() <{value = dense_resource<__elided__> :
        tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc5)

```

```

9      %5 = top.fusion {
10      %6 = "top.fusionInput"(%arg0) : (tensor<1x2048x512xf32>) ->
    tensor<1x2048x512xf32> loc(#loc6)
11      %7 = "top.fusionInput"(%0) : (tensor<1x2048x2048xf32>) ->
    tensor<1x2048x2048xf32> loc(#loc6)
12      %8 = "top.fusionInput"(%1) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32>
    loc(#loc6)
13      %9 = "top.fusionInput"(%2) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
14      %10 = "top.fusionInput"(%3) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
15      %11 = "top.fusionInput"(%4) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
16      %12 = top.tile {
17      %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
    tensor<1x2048x512xf32> loc(#loc7)
18      %19 = "top.tileInput"(%11) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc7)
19      %20 = "top.matmul"(%18, %19) <{transB = false}> :
    (tensor<1x2048x512xf32>, tensor<1x512x1024xf32>) -> tensor<1x2048x1024xf32>
    loc(#loc7)
20      %21 = "top.tileOutput"(%20) : (tensor<1x2048x1024xf32>) ->
    tensor<1x2048x1024xf32> loc(#loc7)
21      top.return %21 : tensor<1x2048x1024xf32> loc(#loc7)
22      } : tensor<1x2048x1024xf32> loc(#loc7)
23      %13 = top.tile {
24      %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
    tensor<1x2048x512xf32> loc(#loc8)
25      %19 = "top.tileInput"(%10) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc8)
26      %20 = "top.matmul"(%18, %19) <{transB = false}> :
    (tensor<1x2048x512xf32>, tensor<1x512x1024xf32>) -> tensor<1x2048x1024xf32>
    loc(#loc8)
27      %21 = "top.transpose"(%20) <{perm = [0, 2, 1]}> :
    (tensor<1x2048x1024xf32>) -> tensor<1x1024x2048xf32> loc(#loc9)
28      %22 = "top.tileOutput"(%21) : (tensor<1x1024x2048xf32>) ->
    tensor<1x1024x2048xf32> loc(#loc8)
29      top.return %22 : tensor<1x1024x2048xf32> loc(#loc8)
30      } : tensor<1x1024x2048xf32> loc(#loc8)
31      %14 = top.tile {
32      %18 = "top.tileInput"(%12) : (tensor<1x2048x1024xf32>) ->
    tensor<1x2048x1024xf32> loc(#loc6)
33      %19 = "top.tileInput"(%13) : (tensor<1x1024x2048xf32>) ->
    tensor<1x1024x2048xf32> loc(#loc6)
34      %20 = "top.tileInput"(%7) : (tensor<1x2048x2048xf32>) ->
    tensor<1x2048x2048xf32> loc(#loc6)

```

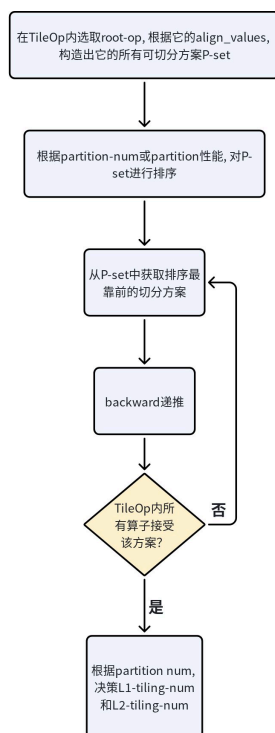
```

35     %21 = "top.tileInput"(%8) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32>
loc(#loc6)
36     %22 = "top.matmul"(%18, %19) <{transB = false}> :
(tensor<1x2048x1024xf32>, tensor<1x1024x2048xf32>) -> tensor<1x2048x2048xf32>
loc(#loc6)
37     %23 = "top.mul"(%22, %21) : (tensor<1x2048x2048xf32>,
tensor<1x1x1xf32>) -> tensor<1x2048x2048xf32> loc(#loc10)
38     %24 = "top.add"(%23, %20) : (tensor<1x2048x2048xf32>,
tensor<1x2048x2048xf32>) -> tensor<1x2048x2048xf32> loc(#loc11)
39     %25 = "top.softmax"(%24) <{axis = 2 : i64}> :
(tensor<1x2048x2048xf32>) -> tensor<1x2048x2048xf32> loc(#loc12)
40     %26 = "top.tileOutput"(%25) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32> loc(#loc6)
41     top.return %26 : tensor<1x2048x2048xf32> loc(#loc6)
42 } : tensor<1x2048x2048xf32> loc(#loc6)
43 %15 = top.tile {
44     %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32> loc(#loc13)
45     %19 = "top.tileInput"(%9) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32> loc(#loc13)
46     %20 = "top.matmul"(%18, %19) <{transB = false}> :
(tensor<1x2048x512xf32>, tensor<1x512x1024xf32>) -> tensor<1x2048x1024xf32>
loc(#loc13)
47     %21 = "top.tileOutput"(%20) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32> loc(#loc13)
48     top.return %21 : tensor<1x2048x1024xf32> loc(#loc13)
49 } : tensor<1x2048x1024xf32> loc(#loc13)
50 %16 = top.tile {
51     %18 = "top.tileInput"(%15) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32> loc(#loc14)
52     %19 = "top.tileInput"(%14) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32> loc(#loc14)
53     %20 = "top.matmul"(%19, %18) <{transB = false}> :
(tensor<1x2048x2048xf32>, tensor<1x2048x1024xf32>) -> tensor<1x2048x1024xf32>
loc(#loc14)
54     %21 = "top.tileOutput"(%20) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32> loc(#loc14)
55     top.return %21 : tensor<1x2048x1024xf32> loc(#loc14)
56 } : tensor<1x2048x1024xf32> loc(#loc14)
57 %17 = "top.fusionOutput"(%16) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32> loc(#loc6)
58     top.return %17 : tensor<1x2048x1024xf32> loc(#loc6)
59 } : tensor<1x2048x1024xf32> loc(#loc6)
60     return %5 : tensor<1x2048x1024xf32> loc(#loc15)
61 } loc(#loc)
62 } loc(#loc)

```

3.4 partition pass

对于一组算子group, 先决策总共可以切多少份: $\text{partition_num} = \text{L2_tiling_num} * \text{core_num} * \text{L1_tiling_num}$

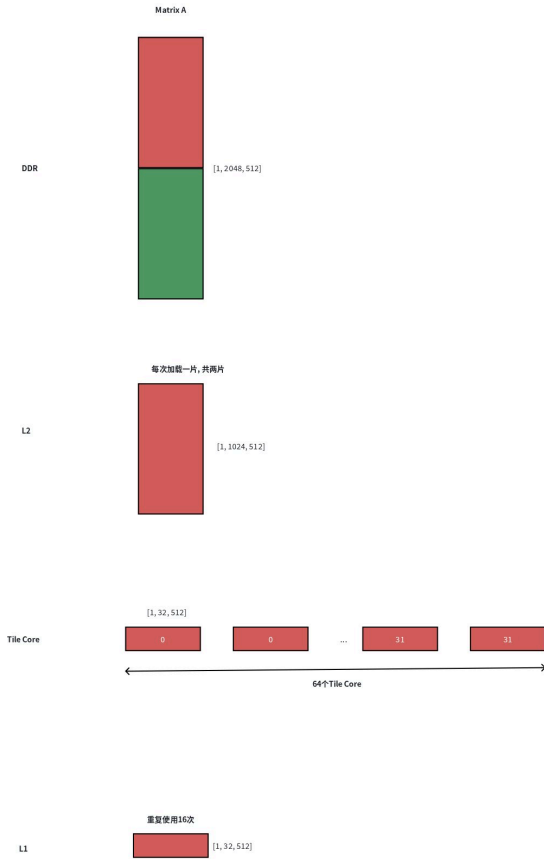


$\text{partition_num} = \text{L2_tiling_num} * \text{core_num} * \text{L1_tiling_num}$, 在已知 partition_num 和 core_num 的情况下, 实现 L2_tiling_num 和 L1_tiling_num 的最优分配

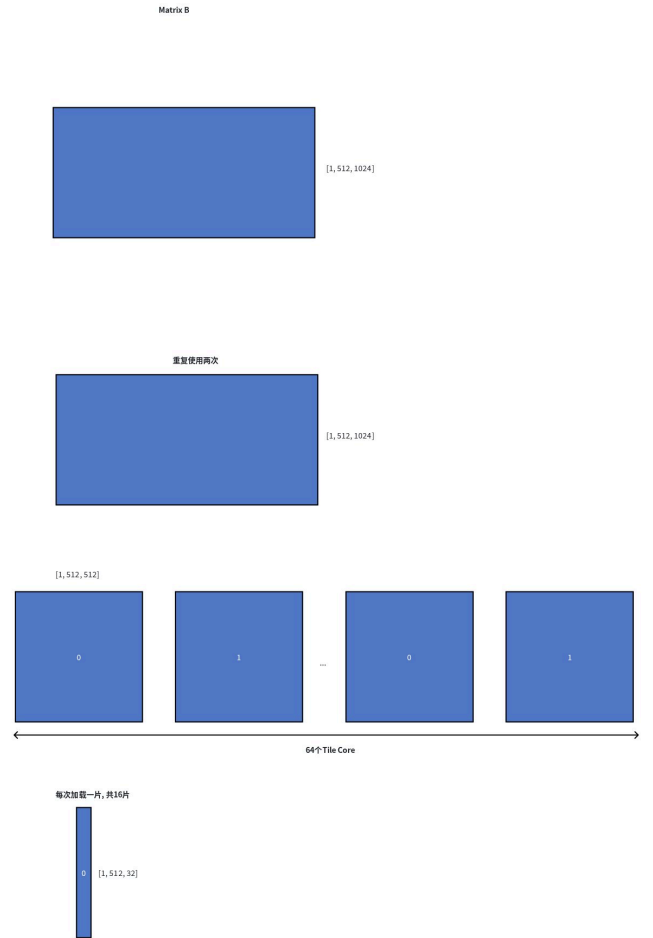
决策方案:

- 隐藏DDR->L2的延迟是最重要的, 所以基于cost-model, 令每个L2切片是DDR->L2和L2->L1 & 计算的latency一致, 即找到最佳的 L2_tiling_num
- L1_tiling_num 能够保证Tile Op所需L1 size不大于物理限制

```
{L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]},
L1_tiling = {replicate_num = 16 : i64, shard_nums = [1, 1, 1]}}
```



```
{L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 16]}}
```



代码块

```
1  #loc = loc(unknown)
2  module attributes {top.core_num = 64 : i64, top.version = "1.0.0"} {
3    func.func @AttentionGraphFixed(%arg0: tensor<1x2048x512xf32> loc(unknown)) -
> tensor<1x2048x1024xf32> {
4      %0 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<1x2048x2048xf32>}> : () -> tensor<1x2048x2048xf32> loc(#loc1)
5      %1 = "top.constant"() <{value = dense<3.125000e-02> : tensor<f32>}> : () -
> tensor<1x1x1xf32> loc(#loc2)
6      %2 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc3)
7      %3 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc4)
8      %4 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc5)
9      %5 = top.fusion {
10      %6 = "top.fusionInput"(%arg0) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32> loc(#loc6)
```

```

11      %7 = "top.fusionInput"(%0) : (tensor<1x2048x2048xf32>) ->
    tensor<1x2048x2048xf32> loc(#loc6)
12      %8 = "top.fusionInput"(%1) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32>
    loc(#loc6)
13      %9 = "top.fusionInput"(%2) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
14      %10 = "top.fusionInput"(%3) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
15      %11 = "top.fusionInput"(%4) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
16      %12 = top.tile {
17      %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
    tensor<1x2048x512xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
    [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
    core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]},
    partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]}}>
    loc(#loc7)
18      %19 = "top.tileInput"(%11) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1,
    1, 16]}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
    core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
    partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}>
    loc(#loc7)
19      %20 = "top.matmul"(%18, %19) <{transB = false}> {align_values = [1,
    32, 32], is_root = true} : (tensor<1x2048x512xf32, {L1_tiling = {replicate_num
    = 16 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64,
    shard_nums = [1, 2, 1]}, core_sharding = {replicate_num = 2 : i64, shard_nums
    = [1, 32, 1]}, partition_config = {replicate_num = 32 : i64, shard_nums = [1,
    64, 1]}}>, tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 1 : i64,
    shard_nums = [1, 1, 16]}, L2_tiling = {replicate_num = 2 : i64, shard_nums =
    [1, 1, 1]}, core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1,
    2]}, partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}>)
    -> tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
    [1, 1, 16], tiling_sizes = [1, 32, 32]}, L2_tiling = {replicate_num = 1 : i64,
    shard_nums = [1, 2, 1], tiling_sizes = [1, 1024, 1024]}, core_sharding =
    {replicate_num = 1 : i64, shard_nums = [1, 32, 2], tiling_sizes = [1, 32,
    512]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64,
    32]}}> loc(#loc7)
20      %21 = "top.tileOutput"(%20) : (tensor<1x2048x1024xf32, {L1_tiling =
    {replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
    32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
    tiling_sizes = [1, 1024, 1024]}, core_sharding = {replicate_num = 1 : i64,
    shard_nums = [1, 32, 2], tiling_sizes = [1, 32, 512]}, partition_config =
    {replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}>) ->
    tensor<1x2048x1024xf32> loc(#loc7)
21      top.return %21 : tensor<1x2048x1024xf32> loc(#loc7)
22      } : tensor<1x2048x1024xf32> loc(#loc7)

```

```

23     %13 = top.tile {
24         %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
        tensor<1x2048x512xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1,
        16, 1]}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
        core_sharding = {replicate_num = 16 : i64, shard_nums = [1, 4, 1]},
        partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]}}>
        loc(#loc8)
25         %19 = "top.tileInput"(%10) : (tensor<1x512x1024xf32>) ->
        tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
        [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 2]},
        core_sharding = {replicate_num = 4 : i64, shard_nums = [1, 1, 16]},
        partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}>
        loc(#loc8)
26         %20 = "top.matmul"(%18, %19) <{transB = false}> :
        (tensor<1x2048x512xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
        [1, 16, 1]}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
        core_sharding = {replicate_num = 16 : i64, shard_nums = [1, 4, 1]},
        partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]}}>,
        tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
        [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 2]},
        core_sharding = {replicate_num = 4 : i64, shard_nums = [1, 1, 16]},
        partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}>) ->
        tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
        [1, 16, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 2]},
        core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 4, 16]},
        partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}>
        loc(#loc8)
27         %21 = "top.transpose"(%20) <{perm = [0, 2, 1]}> {align_values = [1,
        32, 32], is_root = true} : (tensor<1x2048x1024xf32, {L1_tiling =
        {replicate_num = 1 : i64, shard_nums = [1, 16, 1]}, L2_tiling = {replicate_num
        = 1 : i64, shard_nums = [1, 1, 2]}, core_sharding = {replicate_num = 1 : i64,
        shard_nums = [1, 4, 16]}, partition_config = {replicate_num = 1 : i64,
        shard_nums = [1, 64, 32]}}>) -> tensor<1x1024x2048xf32, {L1_tiling =
        {replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
        32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
        tiling_sizes = [1, 512, 2048]}, core_sharding = {replicate_num = 1 : i64,
        shard_nums = [1, 16, 4], tiling_sizes = [1, 32, 512]}, partition_config =
        {replicate_num = 1 : i64, shard_nums = [1, 32, 64]}}> loc(#loc9)
28         %22 = "top.tileOutput"(%21) : (tensor<1x1024x2048xf32, {L1_tiling =
        {replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
        32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
        tiling_sizes = [1, 512, 2048]}, core_sharding = {replicate_num = 1 : i64,
        shard_nums = [1, 16, 4], tiling_sizes = [1, 32, 512]}, partition_config =
        {replicate_num = 1 : i64, shard_nums = [1, 32, 64]}}>) ->
        tensor<1x1024x2048xf32> loc(#loc8)
29         top.return %22 : tensor<1x1024x2048xf32> loc(#loc8)
30     } : tensor<1x1024x2048xf32> loc(#loc8)

```

```

31         %14 = top.tile {
32             %18 = "top.tileInput"(%12) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>
loc(#loc6)
33             %19 = "top.tileInput"(%13) : (tensor<1x1024x2048xf32>) ->
tensor<1x1024x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 64 : i64, shard_nums = [1, 1, 1]},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 1]}}>
loc(#loc6)
34             %20 = "top.tileInput"(%7) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>
loc(#loc6)
35             %21 = "top.tileInput"(%8) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32,
{L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, L2_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, core_sharding =
{replicate_num = 64 : i64, shard_nums = [1, 1, 1]}, partition_config =
{replicate_num = 64 : i64, shard_nums = [1, 1, 1]}}> loc(#loc6)
36             %22 = "top.matmul"(%18, %19) <{transB = false}> :
(tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>,
tensor<1x1024x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 64 : i64, shard_nums = [1, 1, 1]},
partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 1]}}>) ->
tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]},
core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]},
partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>
loc(#loc6)
37             %23 = "top.mul"(%22, %21) : (tensor<1x2048x2048xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num
= 1 : i64, shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 64, 1]}}>, tensor<1x1x1xf32, {L1_tiling = {replicate_num = 1
: i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64,
shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 64 : i64, shard_nums
= [1, 1, 1]}, partition_config = {replicate_num = 64 : i64, shard_nums = [1,
1, 1]}}>) -> tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64,

```

```
shard_nums = [1, 1, 1]], L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}> loc(#loc10)
```

```
38      %24 = "top.add"(%23, %20) : (tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>, tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>) -> tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 32, 2048]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 2048, 2048]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1], tiling_sizes = [1, 32, 2048]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}> loc(#loc11)
```

```
39      %25 = "top.softmax"(%24) <{axis = 2 : i64}> {align_values = [1, 32, 32], is_root = true} : (tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 32, 2048]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 2048, 2048]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1], tiling_sizes = [1, 32, 2048]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>) -> tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 32, 2048]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 2048, 2048]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1], tiling_sizes = [1, 32, 2048]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}> loc(#loc12)
```

```
40      %26 = "top.tileOutput"(%25) : (tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 32, 2048]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 2048, 2048]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 64, 1], tiling_sizes = [1, 32, 2048]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>) -> tensor<1x2048x2048xf32> loc(#loc6)
```

```
41      top.return %26 : tensor<1x2048x2048xf32> loc(#loc6)
```

```
42      } : tensor<1x2048x2048xf32> loc(#loc6)
```

```
43      %15 = top.tile {
```

```
44      %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) -> tensor<1x2048x512xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]}, core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]}, partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]}}> loc(#loc13)
```

```

45      %19 = "top.tileInput"(%9) : (tensor<1x512x1024xf32>) ->
      tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums = [1,
      1, 16]}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
      core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
      partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}>
      loc(#loc13)
46      %20 = "top.matmul"(%18, %19) <{transB = false}> {align_values = [1,
      32, 32], is_root = true} : (tensor<1x2048x512xf32, {L1_tiling = {replicate_num
      = 16 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64,
      shard_nums = [1, 2, 1]}, core_sharding = {replicate_num = 2 : i64, shard_nums
      = [1, 32, 1]}, partition_config = {replicate_num = 32 : i64, shard_nums = [1,
      64, 1]}}>, tensor<1x512x1024xf32, {L1_tiling = {replicate_num = 1 : i64,
      shard_nums = [1, 1, 16]}, L2_tiling = {replicate_num = 2 : i64, shard_nums =
      [1, 1, 1]}, core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1,
      2]}, partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}>)
      -> tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
      [1, 1, 16], tiling_sizes = [1, 32, 32]}, L2_tiling = {replicate_num = 1 : i64,
      shard_nums = [1, 2, 1], tiling_sizes = [1, 1024, 1024]}, core_sharding =
      {replicate_num = 1 : i64, shard_nums = [1, 32, 2], tiling_sizes = [1, 32,
      512]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64,
      32]}}> loc(#loc13)
47      %21 = "top.tileOutput"(%20) : (tensor<1x2048x1024xf32, {L1_tiling =
      {replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
      32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
      tiling_sizes = [1, 1024, 1024]}, core_sharding = {replicate_num = 1 : i64,
      shard_nums = [1, 32, 2], tiling_sizes = [1, 32, 512]}, partition_config =
      {replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}>) ->
      tensor<1x2048x1024xf32> loc(#loc13)
48      top.return %21 : tensor<1x2048x1024xf32> loc(#loc13)
49      } : tensor<1x2048x1024xf32> loc(#loc13)
50      %16 = top.tile {
51      %18 = "top.tileInput"(%15) : (tensor<1x2048x1024xf32>) ->
      tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
      [1, 1, 16]}, L2_tiling = {replicate_num = 2 : i64, shard_nums = [1, 1, 1]},
      core_sharding = {replicate_num = 32 : i64, shard_nums = [1, 1, 2]},
      partition_config = {replicate_num = 64 : i64, shard_nums = [1, 1, 32]}}>
      loc(#loc14)
52      %19 = "top.tileInput"(%14) : (tensor<1x2048x2048xf32>) ->
      tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 16 : i64, shard_nums =
      [1, 1, 1]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1]},
      core_sharding = {replicate_num = 2 : i64, shard_nums = [1, 32, 1]},
      partition_config = {replicate_num = 32 : i64, shard_nums = [1, 64, 1]}}>
      loc(#loc14)
53      %20 = "top.matmul"(%19, %18) <{transB = false}> {align_values = [1,
      32, 32], is_root = true} : (tensor<1x2048x2048xf32, {L1_tiling =
      {replicate_num = 16 : i64, shard_nums = [1, 1, 1]}, L2_tiling = {replicate_num
      = 1 : i64, shard_nums = [1, 2, 1]}, core_sharding = {replicate_num = 2 : i64,

```

```

shard_nums = [1, 32, 1]], partition_config = {replicate_num = 32 : i64,
shard_nums = [1, 64, 1]}}>, tensor<1x2048x1024xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 16]}, L2_tiling = {replicate_num
= 2 : i64, shard_nums = [1, 1, 1]}, core_sharding = {replicate_num = 32 : i64,
shard_nums = [1, 1, 2]}, partition_config = {replicate_num = 64 : i64,
shard_nums = [1, 1, 32]}}>) -> tensor<1x2048x1024xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
tiling_sizes = [1, 1024, 1024]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 32, 2], tiling_sizes = [1, 32, 512]}, partition_config =
{replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}> loc(#loc14)
54      %21 = "top.tileOutput"(%20) : (tensor<1x2048x1024xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
tiling_sizes = [1, 1024, 1024]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 32, 2], tiling_sizes = [1, 32, 512]}, partition_config =
{replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}>) ->
tensor<1x2048x1024xf32> loc(#loc14)
55      top.return %21 : tensor<1x2048x1024xf32> loc(#loc14)
56      } : tensor<1x2048x1024xf32> loc(#loc14)
57      %17 = "top.fusionOutput"(%16) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32> loc(#loc6)
58      top.return %17 : tensor<1x2048x1024xf32> loc(#loc6)
59      } : tensor<1x2048x1024xf32> loc(#loc6)
60      return %5 : tensor<1x2048x1024xf32> loc(#loc15)
61  } loc(#loc)
62 } loc(#loc)
63

```

3.5 top-to-linalg pass

1. 所有计算算子下降到对应的linalg named op
2. 没有对应的linalg named op, 则在linalg_ext中扩充, 并实现TilingInterface

代码块

```

1  #loc = loc(unknown)
2  module attributes {top.core_num = 64 : i64, top.version = "1.0.0"} {
3    func.func @AttentionGraphFixed(%arg0: tensor<1x2048x512xf32> loc(unknown)) -
> tensor<1x2048x1024xf32> {
4      %0 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<1x2048x2048xf32>}> : () -> tensor<1x2048x2048xf32> loc(#loc1)
5      %1 = "top.constant"() <{value = dense<3.125000e-02> : tensor<f32>}> : () -
> tensor<1x1x1xf32> loc(#loc2)

```

```

6      %2 = "top.constant"() <{value = dense_resource<__elided__> :
    tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc3)
7      %3 = "top.constant"() <{value = dense_resource<__elided__> :
    tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc4)
8      %4 = "top.constant"() <{value = dense_resource<__elided__> :
    tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc5)
9      %5 = top.fusion {
10         %6 = "top.fusionInput"(%arg0) : (tensor<1x2048x512xf32>) ->
    tensor<1x2048x512xf32> loc(#loc6)
11         %7 = "top.fusionInput"(%0) : (tensor<1x2048x2048xf32>) ->
    tensor<1x2048x2048xf32> loc(#loc6)
12         %8 = "top.fusionInput"(%1) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32>
    loc(#loc6)
13         %9 = "top.fusionInput"(%2) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
14         %10 = "top.fusionInput"(%3) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
15         %11 = "top.fusionInput"(%4) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
16         %12 = top.tile {
17             %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
    tensor<1x2048x512xf32, {}> loc(#loc7)
18             %19 = "top.tileInput"(%11) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32, {}> loc(#loc7)
19             %20 = tensor.empty() : tensor<1x2048x1024xf32, {L1_tiling =
    {replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
    32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
    tiling_sizes = [1, 1024, 1024]}, core_sharding = {replicate_num = 1 : i64,
    shard_nums = [1, 32, 2], tiling_sizes = [1, 32, 512]}, partition_config =
    {replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}> loc(#loc7)
20             %21 = linalg.batch_matmul {is_root = true} ins(%18, %19 :
    tensor<1x2048x512xf32, {}>, tensor<1x512x1024xf32, {}>) outs(%20 :
    tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
    [1, 1, 16], tiling_sizes = [1, 32, 32]}, L2_tiling = {replicate_num = 1 : i64,
    shard_nums = [1, 2, 1], tiling_sizes = [1, 1024, 1024]}, core_sharding =
    {replicate_num = 1 : i64, shard_nums = [1, 32, 2], tiling_sizes = [1, 32,
    512]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64,
    32]}}>) -> tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64,
    shard_nums = [1, 1, 16], tiling_sizes = [1, 32, 32]}, L2_tiling =
    {replicate_num = 1 : i64, shard_nums = [1, 2, 1], tiling_sizes = [1, 1024,
    1024]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 32, 2],
    tiling_sizes = [1, 32, 512]}, partition_config = {replicate_num = 1 : i64,
    shard_nums = [1, 64, 32]}}> loc(#loc7)
21         %22 = "top.tileOutput"(%21) : (tensor<1x2048x1024xf32, {L1_tiling =
    {replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
    32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
    tiling_sizes = [1, 1024, 1024]}, core_sharding = {replicate_num = 1 : i64,

```

```

shard_nums = [1, 32, 2], tiling_sizes = [1, 32, 512]], partition_config =
{replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}>) ->
tensor<1x2048x1024xf32> loc(#loc7)
22         top.return %22 : tensor<1x2048x1024xf32> loc(#loc7)
23     } : tensor<1x2048x1024xf32> loc(#loc7)
24     %13 = top.tile {
25         %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {}> loc(#loc8)
26         %19 = "top.tileInput"(%10) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}> loc(#loc8)
27         %20 = tensor.empty() : tensor<1x2048x1024xf32> loc(#loc8)
28         %21 = linalg.batch_matmul ins(%18, %19 : tensor<1x2048x512xf32, {}>,
tensor<1x512x1024xf32, {}>) outs(%20 : tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32> loc(#loc8)
29         %22 = tensor.empty() : tensor<1x1024x2048xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
tiling_sizes = [1, 512, 2048]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 16, 4], tiling_sizes = [1, 32, 512]}, partition_config =
{replicate_num = 1 : i64, shard_nums = [1, 32, 64]}}> loc(#loc9)
30         %transposed = linalg.transpose ins(%21 : tensor<1x2048x1024xf32>)
outs(%22 : tensor<1x1024x2048xf32, {L1_tiling = {replicate_num = 1 : i64,
shard_nums = [1, 1, 16], tiling_sizes = [1, 32, 32]}, L2_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 2, 1], tiling_sizes = [1, 512,
2048]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 16, 4],
tiling_sizes = [1, 32, 512]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 32, 64]}}>) permutation = [0, 2, 1] {is_root = true}
loc(#loc9)
31         %23 = "top.tileOutput"(%transposed) : (tensor<1x1024x2048xf32,
{L1_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes =
[1, 32, 32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
tiling_sizes = [1, 512, 2048]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 16, 4], tiling_sizes = [1, 32, 512]}, partition_config =
{replicate_num = 1 : i64, shard_nums = [1, 32, 64]}}>) ->
tensor<1x1024x2048xf32> loc(#loc8)
32         top.return %23 : tensor<1x1024x2048xf32> loc(#loc8)
33     } : tensor<1x1024x2048xf32> loc(#loc8)
34     %14 = top.tile {
35         %18 = "top.tileInput"(%12) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {}> loc(#loc6)
36         %19 = "top.tileInput"(%13) : (tensor<1x1024x2048xf32>) ->
tensor<1x1024x2048xf32, {}> loc(#loc6)
37         %20 = "top.tileInput"(%7) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {}> loc(#loc6)
38         %21 = "top.tileInput"(%8) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32,
{}> loc(#loc6)
39         %22 = tensor.empty() : tensor<1x2048x2048xf32> loc(#loc6)

```

```

40      %23 = linalg.batch_matmul ins(%18, %19 : tensor<1x2048x1024xf32, {}>,
    tensor<1x1024x2048xf32, {}>) outs(%22 : tensor<1x2048x2048xf32>) ->
    tensor<1x2048x2048xf32> loc(#loc6)
41      %24 = tensor.empty() : tensor<1x2048x2048xf32> loc(#loc10)
42      %25 = linalg_ext.mul ins(%23, %21 : tensor<1x2048x2048xf32>,
    tensor<1x1x1xf32, {}>) outs(%24 : tensor<1x2048x2048xf32>) ->
    tensor<1x2048x2048xf32> loc(#loc10)
43      %26 = tensor.empty() : tensor<1x2048x2048xf32> loc(#loc11)
44      %27 = linalg.add ins(%25, %20 : tensor<1x2048x2048xf32>,
    tensor<1x2048x2048xf32, {}>) outs(%26 : tensor<1x2048x2048xf32>) ->
    tensor<1x2048x2048xf32> loc(#loc11)
45      %28 = tensor.empty() : tensor<1x2048x2048xf32, {L1_tiling =
    {replicate_num = 1 : i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 32,
    2048]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1],
    tiling_sizes = [1, 2048, 2048]}, core_sharding = {replicate_num = 1 : i64,
    shard_nums = [1, 64, 1], tiling_sizes = [1, 32, 2048]}, partition_config =
    {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}> loc(#loc12)
46      %29 = linalg.softmax {is_root = true} dimension(2) ins(%27 :
    tensor<1x2048x2048xf32>) outs(%28 : tensor<1x2048x2048xf32, {L1_tiling =
    {replicate_num = 1 : i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 32,
    2048]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1],
    tiling_sizes = [1, 2048, 2048]}, core_sharding = {replicate_num = 1 : i64,
    shard_nums = [1, 64, 1], tiling_sizes = [1, 32, 2048]}, partition_config =
    {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>) ->
    tensor<1x2048x2048xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
    [1, 1, 1], tiling_sizes = [1, 32, 2048]}, L2_tiling = {replicate_num = 1 :
    i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 2048, 2048]}, core_sharding =
    {replicate_num = 1 : i64, shard_nums = [1, 64, 1], tiling_sizes = [1, 32,
    2048]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64,
    1]}}> loc(#loc12)
47      %30 = "top.tileOutput"(%29) : (tensor<1x2048x2048xf32, {L1_tiling =
    {replicate_num = 1 : i64, shard_nums = [1, 1, 1], tiling_sizes = [1, 32,
    2048]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 1, 1],
    tiling_sizes = [1, 2048, 2048]}, core_sharding = {replicate_num = 1 : i64,
    shard_nums = [1, 64, 1], tiling_sizes = [1, 32, 2048]}, partition_config =
    {replicate_num = 1 : i64, shard_nums = [1, 64, 1]}}>) ->
    tensor<1x2048x2048xf32> loc(#loc6)
48      top.return %30 : tensor<1x2048x2048xf32> loc(#loc6)
49      } : tensor<1x2048x2048xf32> loc(#loc6)
50      %15 = top.tile {
51      %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
    tensor<1x2048x512xf32, {}> loc(#loc13)
52      %19 = "top.tileInput"(%9) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32, {}> loc(#loc13)
53      %20 = tensor.empty() : tensor<1x2048x1024xf32, {L1_tiling =
    {replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
    32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],

```

```

tiling_sizes = [1, 1024, 1024]], core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 32, 2], tiling_sizes = [1, 32, 512]], partition_config =
{replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}> loc(#loc13)
54      %21 = linalg.batch_matmul {is_root = true} ins(%18, %19 :
tensor<1x2048x512xf32, {}>, tensor<1x512x1024xf32, {}>) outs(%20 :
tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 16], tiling_sizes = [1, 32, 32]}, L2_tiling = {replicate_num = 1 : i64,
shard_nums = [1, 2, 1], tiling_sizes = [1, 1024, 1024]}, core_sharding =
{replicate_num = 1 : i64, shard_nums = [1, 32, 2], tiling_sizes = [1, 32,
512]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64,
32]}}>) -> tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64,
shard_nums = [1, 1, 16], tiling_sizes = [1, 32, 32]}, L2_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 2, 1], tiling_sizes = [1, 1024,
1024]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 32, 2],
tiling_sizes = [1, 32, 512]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 64, 32]}}> loc(#loc13)
55      %22 = "top.tileOutput"(%21) : (tensor<1x2048x1024xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
tiling_sizes = [1, 1024, 1024]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 32, 2], tiling_sizes = [1, 32, 512]}, partition_config =
{replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}>) ->
tensor<1x2048x1024xf32> loc(#loc13)
56      top.return %22 : tensor<1x2048x1024xf32> loc(#loc13)
57      } : tensor<1x2048x1024xf32> loc(#loc13)
58      %16 = top.tile {
59      %18 = "top.tileInput"(%15) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {}> loc(#loc14)
60      %19 = "top.tileInput"(%14) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {}> loc(#loc14)
61      %20 = tensor.empty() : tensor<1x2048x1024xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
tiling_sizes = [1, 1024, 1024]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 32, 2], tiling_sizes = [1, 32, 512]}, partition_config =
{replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}> loc(#loc14)
62      %21 = linalg.batch_matmul {is_root = true} ins(%19, %18 :
tensor<1x2048x2048xf32, {}>, tensor<1x2048x1024xf32, {}>) outs(%20 :
tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64, shard_nums =
[1, 1, 16], tiling_sizes = [1, 32, 32]}, L2_tiling = {replicate_num = 1 : i64,
shard_nums = [1, 2, 1], tiling_sizes = [1, 1024, 1024]}, core_sharding =
{replicate_num = 1 : i64, shard_nums = [1, 32, 2], tiling_sizes = [1, 32,
512]}, partition_config = {replicate_num = 1 : i64, shard_nums = [1, 64,
32]}}>) -> tensor<1x2048x1024xf32, {L1_tiling = {replicate_num = 1 : i64,
shard_nums = [1, 1, 16], tiling_sizes = [1, 32, 32]}, L2_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 2, 1], tiling_sizes = [1, 1024,
1024]}, core_sharding = {replicate_num = 1 : i64, shard_nums = [1, 32, 2],

```

```

tiling_sizes = [1, 32, 512]}, partition_config = {replicate_num = 1 : i64,
shard_nums = [1, 64, 32]}}> loc(#loc14)
63      %22 = "top.tileOutput"(%21) : (tensor<1x2048x1024xf32, {L1_tiling =
{replicate_num = 1 : i64, shard_nums = [1, 1, 16], tiling_sizes = [1, 32,
32]}, L2_tiling = {replicate_num = 1 : i64, shard_nums = [1, 2, 1],
tiling_sizes = [1, 1024, 1024]}, core_sharding = {replicate_num = 1 : i64,
shard_nums = [1, 32, 2], tiling_sizes = [1, 32, 512]}, partition_config =
{replicate_num = 1 : i64, shard_nums = [1, 64, 32]}}>) ->
tensor<1x2048x1024xf32> loc(#loc14)
64      top.return %22 : tensor<1x2048x1024xf32> loc(#loc14)
65      } : tensor<1x2048x1024xf32> loc(#loc14)
66      %17 = "top.fusionOutput"(%16) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32> loc(#loc6)
67      top.return %17 : tensor<1x2048x1024xf32> loc(#loc6)
68      } : tensor<1x2048x1024xf32> loc(#loc6)
69      return %5 : tensor<1x2048x1024xf32> loc(#loc15)
70      } loc(#loc)
71      } loc(#loc)
72

```

3.6 linalg-tile-and-fuse pass

1. 根据L2_tiling_num, core_num和L1_tiling_num, 计算L2_tile_size, sharding_size, L1_tile_size
2. 调用linalg Tiling interface, 完成到scf for的下降(scf.forall表示空间并行, scf.for表示时间片串行)

规则形状

代码块

```

1  #loc = loc(unknown)
2  module attributes {top.core_num = 64 : i64, top.version = "1.0.0"} {
3    func.func @AttentionGraphFixed(%arg0: tensor<1x2048x512xf32> loc(unknown)) -
> tensor<1x2048x1024xf32> {
4      %0 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<1x2048x2048xf32>}> : () -> tensor<1x2048x2048xf32> loc(#loc1)
5      %1 = "top.constant"() <{value = dense<3.125000e-02> : tensor<f32>}> : () -
> tensor<1x1x1xf32> loc(#loc2)
6      %2 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc3)
7      %3 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc4)
8      %4 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32> loc(#loc5)

```

```

9      %5 = top.fusion {
10      %6 = "top.fusionInput"(%arg0) : (tensor<1x2048x512xf32>) ->
    tensor<1x2048x512xf32> loc(#loc6)
11      %7 = "top.fusionInput"(%0) : (tensor<1x2048x2048xf32>) ->
    tensor<1x2048x2048xf32> loc(#loc6)
12      %8 = "top.fusionInput"(%1) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32>
    loc(#loc6)
13      %9 = "top.fusionInput"(%2) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
14      %10 = "top.fusionInput"(%3) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
15      %11 = "top.fusionInput"(%4) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32> loc(#loc6)
16      %12 = top.tile {
17      %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
    tensor<1x2048x512xf32, {}> loc(#loc7)
18      %19 = "top.tileInput"(%11) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32, {}> loc(#loc7)
19      %20 = tensor.empty() : tensor<1x2048x1024xf32, {}> loc(#loc7)
20      %c0 = arith.constant 0 : index loc(#loc7)
21      %c0_0 = arith.constant 0 : index loc(#loc7)
22      %c0_1 = arith.constant 0 : index loc(#loc7)
23      %c1 = arith.constant 1 : index loc(#loc7)
24      %c2048 = arith.constant 2048 : index loc(#loc7)
25      %c1024 = arith.constant 1024 : index loc(#loc7)
26      %c1_2 = arith.constant 1 : index loc(#loc7)
27      %c1024_3 = arith.constant 1024 : index loc(#loc7)
28      %c1024_4 = arith.constant 1024 : index loc(#loc7)
29      %21 = scf.for %arg1 = %c0 to %c1 step %c1_2 iter_args(%arg2 = %20) ->
    (tensor<1x2048x1024xf32, {}>) {
30      %23 = scf.for %arg3 = %c0_0 to %c2048 step %c1024_3 iter_args(%arg4
    = %arg2) -> (tensor<1x2048x1024xf32, {}>) {
31      %24 = scf.for %arg5 = %c0_1 to %c1024 step %c1024_4
    iter_args(%arg6 = %arg4) -> (tensor<1x2048x1024xf32, {}>) {
32      %25 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
    tensor<1x2048x512xf32, {}> loc(#loc7)
33      %extracted_slice = tensor.extract_slice %18[%arg1, %arg3, 0] [1,
    1024, 512] [1, 1, 1] : tensor<1x2048x512xf32, {}> to tensor<1x1024x512xf32,
    {}> loc(#loc7)
34      %26 = "top.tileInput"(%11) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32, {}> loc(#loc7)
35      %extracted_slice_5 = tensor.extract_slice %19[%arg1, 0, %arg5]
    [1, 512, 1024] [1, 1, 1] : tensor<1x512x1024xf32, {}> to
    tensor<1x512x1024xf32, {}> loc(#loc7)
36      %extracted_slice_6 = tensor.extract_slice %arg6[%arg1, %arg3,
    %arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x2048x1024xf32, {}> to
    tensor<1x1024x1024xf32, {}> loc(#loc7)

```

```

37         %27 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, 1024,
1024) step (1, 32, 512) shared_outs(%arg10 = %extracted_slice_6) ->
(tensor<1x1024x1024xf32, {}>) {
38             %extracted_slice_7 = tensor.extract_slice
%extracted_slice[%arg7, %arg8, 0] [1, 32, 512] [1, 1, 1] :
tensor<1x1024x512xf32, {}> to tensor<1x32x512xf32, {}> loc(#loc7)
39             %extracted_slice_8 = tensor.extract_slice
%extracted_slice_5[%arg7, 0, %arg9] [1, 512, 512] [1, 1, 1] :
tensor<1x512x1024xf32, {}> to tensor<1x512x512xf32, {}> loc(#loc7)
40             %extracted_slice_9 = tensor.extract_slice %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x1024x1024xf32, {}> to
tensor<1x32x512xf32, {}> loc(#loc7)
41             %c0_10 = arith.constant 0 : index loc(#loc7)
42             %c0_11 = arith.constant 0 : index loc(#loc7)
43             %c0_12 = arith.constant 0 : index loc(#loc7)
44             %c1_13 = arith.constant 1 : index loc(#loc7)
45             %c32 = arith.constant 32 : index loc(#loc7)
46             %c512 = arith.constant 512 : index loc(#loc7)
47             %c1_14 = arith.constant 1 : index loc(#loc7)
48             %c32_15 = arith.constant 32 : index loc(#loc7)
49             %c32_16 = arith.constant 32 : index loc(#loc7)
50             %28 = scf.for %arg11 = %c0_10 to %c1_13 step %c1_14
iter_args(%arg12 = %extracted_slice_9) -> (tensor<1x32x512xf32, {}>) {
51                 %29 = scf.for %arg13 = %c0_11 to %c32 step %c32_15
iter_args(%arg14 = %arg12) -> (tensor<1x32x512xf32, {}>) {
52                     %30 = scf.for %arg15 = %c0_12 to %c512 step %c32_16
iter_args(%arg16 = %arg14) -> (tensor<1x32x512xf32, {}>) {
53                         %extracted_slice_17 = tensor.extract_slice
%extracted_slice_7[%arg11, %arg13, 0] [1, 32, 512] [1, 1, 1] :
tensor<1x32x512xf32, {}> to tensor<1x32x512xf32, {}> loc(#loc7)
54                         %extracted_slice_18 = tensor.extract_slice
%extracted_slice_8[%arg11, 0, %arg15] [1, 512, 32] [1, 1, 1] :
tensor<1x512x512xf32, {}> to tensor<1x512x32xf32, {}> loc(#loc7)
55                         %extracted_slice_19 = tensor.extract_slice
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x512xf32,
{}> to tensor<1x32x32xf32, {}> loc(#loc7)
56                         %31 = linalg.batch_matmul {is_root = true}
ins(%extracted_slice_17, %extracted_slice_18 : tensor<1x32x512xf32, {}>,
tensor<1x512x32xf32, {}>) outs(%extracted_slice_19 : tensor<1x32x32xf32, {}>) -
> tensor<1x32x32xf32, {}> loc(#loc7)
57                         %inserted_slice_20 = tensor.insert_slice %31 into
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x32xf32, {}>
into tensor<1x32x512xf32, {}> loc(#loc7)
58                         scf.yield %inserted_slice_20 : tensor<1x32x512xf32, {}>
loc(#loc7)
59                     } {tiling_level = "L1_tiling"} loc(#loc7)
60                 scf.yield %30 : tensor<1x32x512xf32, {}> loc(#loc7)

```

```

61         } {tiling_level = "L1_tiling"} loc(#loc7)
62         scf.yield %29 : tensor<1x32x512xf32, {}> loc(#loc7)
63         } {tiling_level = "L1_tiling"} loc(#loc7)
64         scf.forall.in_parallel {
65             tensor.parallel_insert_slice %28 into %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x32x512xf32, {}> into
tensor<1x1024x1024xf32, {}> loc(#loc7)
66         } loc(#loc7)
67         } {tiling_level = "core_sharding"} loc(#loc7)
68         %inserted_slice = tensor.insert_slice %27 into %arg6[%arg1,
%arg3, %arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x1024x1024xf32, {}> into
tensor<1x2048x1024xf32, {}> loc(#loc7)
69         scf.yield %inserted_slice : tensor<1x2048x1024xf32, {}>
loc(#loc7)
70         } {tiling_level = "L2_tiling"} loc(#loc7)
71         scf.yield %24 : tensor<1x2048x1024xf32, {}> loc(#loc7)
72         } {tiling_level = "L2_tiling"} loc(#loc7)
73         scf.yield %23 : tensor<1x2048x1024xf32, {}> loc(#loc7)
74         } {tiling_level = "L2_tiling"} loc(#loc7)
75         %22 = "top.tileOutput"(%21) : (tensor<1x2048x1024xf32, {}>) ->
tensor<1x2048x1024xf32> loc(#loc7)
76         top.return %22 : tensor<1x2048x1024xf32> loc(#loc7)
77         } : tensor<1x2048x1024xf32> loc(#loc7)
78         %13 = top.tile {
79             %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {}> loc(#loc8)
80             %19 = "top.tileInput"(%10) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}> loc(#loc8)
81             %20 = tensor.empty() : tensor<1x2048x1024xf32> loc(#loc8)
82             %21 = tensor.empty() : tensor<1x1024x2048xf32, {}> loc(#loc9)
83             %c0 = arith.constant 0 : index loc(#loc9)
84             %c0_0 = arith.constant 0 : index loc(#loc9)
85             %c0_1 = arith.constant 0 : index loc(#loc9)
86             %c1 = arith.constant 1 : index loc(#loc9)
87             %c1024 = arith.constant 1024 : index loc(#loc9)
88             %c2048 = arith.constant 2048 : index loc(#loc9)
89             %c1_2 = arith.constant 1 : index loc(#loc9)
90             %c512 = arith.constant 512 : index loc(#loc9)
91             %c2048_3 = arith.constant 2048 : index loc(#loc9)
92             %22 = scf.for %arg1 = %c0 to %c1 step %c1_2 iter_args(%arg2 = %21) ->
(tensor<1x1024x2048xf32, {}>) {
93                 %24 = scf.for %arg3 = %c0_0 to %c1024 step %c512 iter_args(%arg4 =
%arg2) -> (tensor<1x1024x2048xf32, {}>) {
94                     %25 = scf.for %arg5 = %c0_1 to %c2048 step %c2048_3
iter_args(%arg6 = %arg4) -> (tensor<1x1024x2048xf32, {}>) {
95                         %26 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {}> loc(#loc8)

```

```

96         %extracted_slice = tensor.extract_slice %18[%arg1, %arg5, 0] [1,
2048, 512] [1, 1, 1] : tensor<1x2048x512xf32, {}> to tensor<1x2048x512xf32,
{}> loc(#loc8)
97         %27 = "top.tileInput"(%10) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}> loc(#loc8)
98         %extracted_slice_4 = tensor.extract_slice %19[%arg1, 0, %arg3]
[1, 512, 512] [1, 1, 1] : tensor<1x512x1024xf32, {}> to tensor<1x512x512xf32,
{}> loc(#loc8)
99         %extracted_slice_5 = tensor.extract_slice %20[%arg1, %arg5,
%arg3] [1, 2048, 512] [1, 1, 1] : tensor<1x2048x1024xf32> to
tensor<1x2048x512xf32> loc(#loc8)
100        %extracted_slice_6 = tensor.extract_slice %arg6[%arg1, %arg3,
%arg5] [1, 512, 2048] [1, 1, 1] : tensor<1x1024x2048xf32, {}> to
tensor<1x512x2048xf32, {}> loc(#loc9)
101        %28 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, 512,
2048) step (1, 32, 512) shared_outs(%arg10 = %extracted_slice_6) ->
(tensor<1x512x2048xf32, {}>) {
102            %extracted_slice_7 = tensor.extract_slice
%extracted_slice[%arg7, %arg9, 0] [1, 512, 512] [1, 1, 1] :
tensor<1x2048x512xf32, {}> to tensor<1x512x512xf32, {}> loc(#loc8)
103            %extracted_slice_8 = tensor.extract_slice
%extracted_slice_4[%arg7, 0, %arg8] [1, 512, 32] [1, 1, 1] :
tensor<1x512x512xf32, {}> to tensor<1x512x32xf32, {}> loc(#loc8)
104            %extracted_slice_9 = tensor.extract_slice
%extracted_slice_5[%arg7, %arg9, %arg8] [1, 512, 32] [1, 1, 1] :
tensor<1x2048x512xf32> to tensor<1x512x32xf32> loc(#loc8)
105            %extracted_slice_10 = tensor.extract_slice %arg10[%arg7,
%arg8, %arg9] [1, 32, 512] [1, 1, 1] : tensor<1x512x2048xf32, {}> to
tensor<1x32x512xf32, {}> loc(#loc9)
106            %c0_11 = arith.constant 0 : index loc(#loc10)
107            %c1_12 = arith.constant 1 : index loc(#loc10)
108            %c0_13 = arith.constant 0 : index loc(#loc9)
109            %c0_14 = arith.constant 0 : index loc(#loc9)
110            %c0_15 = arith.constant 0 : index loc(#loc9)
111            %c1_16 = arith.constant 1 : index loc(#loc9)
112            %c32 = arith.constant 32 : index loc(#loc9)
113            %c512_17 = arith.constant 512 : index loc(#loc9)
114            %c1_18 = arith.constant 1 : index loc(#loc9)
115            %c32_19 = arith.constant 32 : index loc(#loc9)
116            %c32_20 = arith.constant 32 : index loc(#loc9)
117            %29 = scf.for %arg11 = %c0_13 to %c1_16 step %c1_18
iter_args(%arg12 = %extracted_slice_10) -> (tensor<1x32x512xf32, {}>) {
118                %30 = scf.for %arg13 = %c0_14 to %c32 step %c32_19
iter_args(%arg14 = %arg12) -> (tensor<1x32x512xf32, {}>) {
119                    %31 = scf.for %arg15 = %c0_15 to %c512_17 step %c32_20
iter_args(%arg16 = %arg14) -> (tensor<1x32x512xf32, {}>) {

```

```

120         %extracted_slice_21 = tensor.extract_slice
%extracted_slice_7[%arg11, %arg15, 0] [1, 32, 512] [1, 1, 1] :
tensor<1x512x512xf32, {}> to tensor<1x32x512xf32, {}> loc(#loc8)
121         %extracted_slice_22 = tensor.extract_slice
%extracted_slice_8[%arg11, 0, %arg13] [1, 512, 32] [1, 1, 1] :
tensor<1x512x32xf32, {}> to tensor<1x512x32xf32, {}> loc(#loc8)
122         %extracted_slice_23 = tensor.extract_slice
%extracted_slice_9[%arg11, %arg15, %arg13] [1, 32, 32] [1, 1, 1] :
tensor<1x512x32xf32> to tensor<1x32x32xf32> loc(#loc8)
123         %32 = linalg.batch_matmul ins(%extracted_slice_21,
%extracted_slice_22 : tensor<1x32x512xf32, {}>, tensor<1x512x32xf32, {}>)
outs(%extracted_slice_23 : tensor<1x32x32xf32>) -> tensor<1x32x32xf32>
loc(#loc8)
124         %extracted_slice_24 = tensor.extract_slice
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x512xf32,
{}> to tensor<1x32x32xf32, {}> loc(#loc9)
125         %transposed = linalg.transpose ins(%32 :
tensor<1x32x32xf32>) outs(%extracted_slice_24 : tensor<1x32x32xf32, {}>)
permutation = [0, 2, 1] {is_root = true} loc(#loc9)
126         %inserted_slice_25 = tensor.insert_slice %transposed
into %arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] :
tensor<1x32x32xf32, {}> into tensor<1x32x512xf32, {}> loc(#loc9)
127         scf.yield %inserted_slice_25 : tensor<1x32x512xf32, {}>
loc(#loc9)
128         } {tiling_level = "L1_tiling"} loc(#loc9)
129         scf.yield %31 : tensor<1x32x512xf32, {}> loc(#loc9)
130         } {tiling_level = "L1_tiling"} loc(#loc9)
131         scf.yield %30 : tensor<1x32x512xf32, {}> loc(#loc9)
132         } {tiling_level = "L1_tiling"} loc(#loc9)
133         scf.forall.in_parallel {
134             tensor.parallel_insert_slice %29 into %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x32x512xf32, {}> into
tensor<1x512x2048xf32, {}> loc(#loc9)
135             } loc(#loc9)
136             } {tiling_level = "core_sharding"} loc(#loc9)
137             %inserted_slice = tensor.insert_slice %28 into %arg6[%arg1,
%arg3, %arg5] [1, 512, 2048] [1, 1, 1] : tensor<1x512x2048xf32, {}> into
tensor<1x1024x2048xf32, {}> loc(#loc9)
138             scf.yield %inserted_slice : tensor<1x1024x2048xf32, {}>
loc(#loc9)
139             } {tiling_level = "L2_tiling"} loc(#loc9)
140             scf.yield %25 : tensor<1x1024x2048xf32, {}> loc(#loc9)
141             } {tiling_level = "L2_tiling"} loc(#loc9)
142             scf.yield %24 : tensor<1x1024x2048xf32, {}> loc(#loc9)
143             } {tiling_level = "L2_tiling"} loc(#loc9)
144             %23 = "top.tileOutput"(%22) : (tensor<1x1024x2048xf32, {}>) ->
tensor<1x1024x2048xf32> loc(#loc8)

```

```

145         top.return %23 : tensor<1x1024x2048xf32> loc(#loc8)
146     } : tensor<1x1024x2048xf32> loc(#loc8)
147     %14 = top.tile {
148         %18 = "top.tileInput"(%12) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {}> loc(#loc6)
149         %19 = "top.tileInput"(%13) : (tensor<1x1024x2048xf32>) ->
tensor<1x1024x2048xf32, {}> loc(#loc6)
150         %20 = "top.tileInput"(%7) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {}> loc(#loc6)
151         %21 = "top.tileInput"(%8) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32,
{}> loc(#loc6)
152         %22 = tensor.empty() : tensor<1x2048x2048xf32> loc(#loc6)
153         %23 = tensor.empty() : tensor<1x2048x2048xf32> loc(#loc11)
154         %24 = tensor.empty() : tensor<1x2048x2048xf32> loc(#loc12)
155         %25 = tensor.empty() : tensor<1x2048x2048xf32, {}> loc(#loc10)
156         %c0 = arith.constant 0 : index loc(#loc10)
157         %c1 = arith.constant 1 : index loc(#loc10)
158         %c0_0 = arith.constant 0 : index loc(#loc10)
159         %c1_1 = arith.constant 1 : index loc(#loc10)
160         %c0_2 = arith.constant 0 : index loc(#loc10)
161         %c1_3 = arith.constant 1 : index loc(#loc10)
162         %c1_4 = arith.constant 1 : index loc(#loc10)
163         %c2048 = arith.constant 2048 : index loc(#loc10)
164         %c1_5 = arith.constant 1 : index loc(#loc10)
165         %c2048_6 = arith.constant 2048 : index loc(#loc10)
166         %26 = scf.for %arg1 = %c0_2 to %c1_4 step %c1_5 iter_args(%arg2 = %25)
-> (tensor<1x2048x2048xf32, {}>) {
167             %28 = scf.for %arg3 = %c0_2 to %c2048 step %c2048_6 iter_args(%arg4
= %arg2) -> (tensor<1x2048x2048xf32, {}>) {
168                 %29 = "top.tileInput"(%12) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {}> loc(#loc6)
169                 %extracted_slice = tensor.extract_slice %18[%arg1, %arg3, 0] [1,
2048, 1024] [1, 1, 1] : tensor<1x2048x1024xf32, {}> to tensor<1x2048x1024xf32,
{}> loc(#loc6)
170                 %30 = "top.tileInput"(%13) : (tensor<1x1024x2048xf32>) ->
tensor<1x1024x2048xf32, {}> loc(#loc6)
171                 %extracted_slice_7 = tensor.extract_slice %19[%arg1, 0, 0] [1,
1024, 2048] [1, 1, 1] : tensor<1x1024x2048xf32, {}> to tensor<1x1024x2048xf32,
{}> loc(#loc6)
172                 %extracted_slice_8 = tensor.extract_slice %22[%arg1, %arg3, 0] [1,
2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32> to tensor<1x2048x2048xf32>
loc(#loc6)
173                 %31 = "top.tileInput"(%8) : (tensor<1x1x1xf32>) ->
tensor<1x1x1xf32, {}> loc(#loc6)
174                 %extracted_slice_9 = tensor.extract_slice %21[%arg1, 0, 0] [1, 1,
1] [1, 1, 1] : tensor<1x1x1xf32, {}> to tensor<1x1x1xf32, {}> loc(#loc11)

```

```

175         %extracted_slice_10 = tensor.extract_slice %23[%arg1, %arg3, 0]
[1, 2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32> to tensor<1x2048x2048xf32>
loc(#loc11)
176         %32 = "top.tileInput"(%7) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {}> loc(#loc6)
177         %extracted_slice_11 = tensor.extract_slice %20[%arg1, %arg3, 0]
[1, 2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32, {}> to
tensor<1x2048x2048xf32, {}> loc(#loc12)
178         %extracted_slice_12 = tensor.extract_slice %24[%arg1, %arg3, 0]
[1, 2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32> to tensor<1x2048x2048xf32>
loc(#loc12)
179         %extracted_slice_13 = tensor.extract_slice %arg4[%arg1, %arg3,
%c0_2] [1, 2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32, {}> to
tensor<1x2048x2048xf32, {}> loc(#loc10)
180         %c0_14 = arith.constant 0 : index loc(#loc10)
181         %c1_15 = arith.constant 1 : index loc(#loc10)
182         %c0_16 = arith.constant 0 : index loc(#loc10)
183         %c1_17 = arith.constant 1 : index loc(#loc10)
184         %c0_18 = arith.constant 0 : index loc(#loc10)
185         %c1_19 = arith.constant 1 : index loc(#loc10)
186         %33 = scf.forall (%arg5, %arg6) = (%c0_18, %c0_18) to (1, 2048)
step (1, 32) shared_outs(%arg7 = %extracted_slice_13) ->
(tensor<1x2048x2048xf32, {}>) {
187         %extracted_slice_20 = tensor.extract_slice
%extracted_slice[%arg5, %arg6, 0] [1, 32, 1024] [1, 1, 1] :
tensor<1x2048x1024xf32, {}> to tensor<1x32x1024xf32, {}> loc(#loc6)
188         %extracted_slice_21 = tensor.extract_slice
%extracted_slice_7[%arg5, 0, 0] [1, 1024, 2048] [1, 1, 1] :
tensor<1x1024x2048xf32, {}> to tensor<1x1024x2048xf32, {}> loc(#loc6)
189         %extracted_slice_22 = tensor.extract_slice
%extracted_slice_8[%arg5, %arg6, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x2048x2048xf32> to tensor<1x32x2048xf32> loc(#loc6)
190         %extracted_slice_23 = tensor.extract_slice
%extracted_slice_9[%arg5, 0, 0] [1, 1, 1] [1, 1, 1] : tensor<1x1x1xf32, {}> to
tensor<1x1x1xf32, {}> loc(#loc11)
191         %extracted_slice_24 = tensor.extract_slice
%extracted_slice_10[%arg5, %arg6, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x2048x2048xf32> to tensor<1x32x2048xf32> loc(#loc11)
192         %extracted_slice_25 = tensor.extract_slice
%extracted_slice_11[%arg5, %arg6, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x2048x2048xf32, {}> to tensor<1x32x2048xf32, {}> loc(#loc12)
193         %extracted_slice_26 = tensor.extract_slice
%extracted_slice_12[%arg5, %arg6, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x2048x2048xf32> to tensor<1x32x2048xf32> loc(#loc12)
194         %extracted_slice_27 = tensor.extract_slice %arg7[%arg5, %arg6,
%c0_18] [1, 32, 2048] [1, 1, 1] : tensor<1x2048x2048xf32, {}> to
tensor<1x32x2048xf32, {}> loc(#loc10)

```

```

195         %c0_28 = arith.constant 0 : index loc(#loc10)
196         %c1_29 = arith.constant 1 : index loc(#loc10)
197         %c0_30 = arith.constant 0 : index loc(#loc10)
198         %c1_31 = arith.constant 1 : index loc(#loc10)
199         %c1_32 = arith.constant 1 : index loc(#loc10)
200         %c32 = arith.constant 32 : index loc(#loc10)
201         %c1_33 = arith.constant 1 : index loc(#loc10)
202         %c32_34 = arith.constant 32 : index loc(#loc10)
203         %34 = scf.for %arg8 = %c0_30 to %c1_32 step %c1_33
iter_args(%arg9 = %extracted_slice_27) -> (tensor<1x32x2048xf32, {}>) {
204             %35 = scf.for %arg10 = %c0_30 to %c32 step %c32_34
iter_args(%arg11 = %arg9) -> (tensor<1x32x2048xf32, {}>) {
205                 %extracted_slice_35 = tensor.extract_slice
%extracted_slice_20[%arg8, %arg10, 0] [1, 32, 1024] [1, 1, 1] :
tensor<1x32x1024xf32, {}> to tensor<1x32x1024xf32, {}> loc(#loc6)
206                 %extracted_slice_36 = tensor.extract_slice
%extracted_slice_21[%arg8, 0, 0] [1, 1024, 2048] [1, 1, 1] :
tensor<1x1024x2048xf32, {}> to tensor<1x1024x2048xf32, {}> loc(#loc6)
207                 %extracted_slice_37 = tensor.extract_slice
%extracted_slice_22[%arg8, %arg10, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x32x2048xf32> to tensor<1x32x2048xf32> loc(#loc6)
208                 %36 = linalg.batch_matmul ins(%extracted_slice_35,
%extracted_slice_36 : tensor<1x32x1024xf32, {}>, tensor<1x1024x2048xf32, {}>)
outs(%extracted_slice_37 : tensor<1x32x2048xf32>) -> tensor<1x32x2048xf32>
loc(#loc6)
209                 %extracted_slice_38 = tensor.extract_slice
%extracted_slice_23[%arg8, 0, 0] [1, 1, 1] [1, 1, 1] : tensor<1x1x1xf32, {}>
to tensor<1x1x1xf32, {}> loc(#loc11)
210                 %extracted_slice_39 = tensor.extract_slice
%extracted_slice_24[%arg8, %arg10, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x32x2048xf32> to tensor<1x32x2048xf32> loc(#loc11)
211                 %37 = linalg_ext.mul ins(%36, %extracted_slice_38 :
tensor<1x32x2048xf32>, tensor<1x1x1xf32, {}>) outs(%extracted_slice_39 :
tensor<1x32x2048xf32>) -> tensor<1x32x2048xf32> loc(#loc11)
212                 %extracted_slice_40 = tensor.extract_slice
%extracted_slice_25[%arg8, %arg10, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x32x2048xf32, {}> to tensor<1x32x2048xf32, {}> loc(#loc12)
213                 %extracted_slice_41 = tensor.extract_slice
%extracted_slice_26[%arg8, %arg10, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x32x2048xf32> to tensor<1x32x2048xf32> loc(#loc12)
214                 %38 = linalg.add ins(%37, %extracted_slice_40 :
tensor<1x32x2048xf32>, tensor<1x32x2048xf32, {}>) outs(%extracted_slice_41 :
tensor<1x32x2048xf32>) -> tensor<1x32x2048xf32> loc(#loc12)
215                 %extracted_slice_42 = tensor.extract_slice %arg11[%arg8,
%arg10, %c0_30] [1, 32, 2048] [1, 1, 1] : tensor<1x32x2048xf32, {}> to
tensor<1x32x2048xf32, {}> loc(#loc10)

```

```

216         %39 = linalg.softmax {is_root = true} dimension(2) ins(%38 :
    tensor<1x32x2048xf32>) outs(%extracted_slice_42 : tensor<1x32x2048xf32, {}>) -
    > tensor<1x32x2048xf32, {}> loc(#loc10)
217         %inserted_slice_43 = tensor.insert_slice %39 into
    %arg11[%arg8, %arg10, %c0_30] [1, 32, 2048] [1, 1, 1] : tensor<1x32x2048xf32,
    {}> into tensor<1x32x2048xf32, {}> loc(#loc10)
218         scf.yield %inserted_slice_43 : tensor<1x32x2048xf32, {}>
    loc(#loc10)
219         } {tiling_level = "L1_tiling"} loc(#loc10)
220         scf.yield %35 : tensor<1x32x2048xf32, {}> loc(#loc10)
221         } {tiling_level = "L1_tiling"} loc(#loc10)
222         scf.forall.in_parallel {
223             tensor.parallel_insert_slice %34 into %arg7[%arg5, %arg6,
    %c0_18] [1, 32, 2048] [1, 1, 1] : tensor<1x32x2048xf32, {}> into
    tensor<1x2048x2048xf32, {}> loc(#loc10)
224             } loc(#loc10)
225             } {tiling_level = "core_sharding"} loc(#loc10)
226             %inserted_slice = tensor.insert_slice %33 into %arg4[%arg1, %arg3,
    %c0_2] [1, 2048, 2048] [1, 1, 1] : tensor<1x2048x2048xf32, {}> into
    tensor<1x2048x2048xf32, {}> loc(#loc10)
227             scf.yield %inserted_slice : tensor<1x2048x2048xf32, {}> loc(#loc10)
228             } {tiling_level = "L2_tiling"} loc(#loc10)
229             scf.yield %28 : tensor<1x2048x2048xf32, {}> loc(#loc10)
230             } {tiling_level = "L2_tiling"} loc(#loc10)
231             %27 = "top.tileOutput"(%26) : (tensor<1x2048x2048xf32, {}>) ->
    tensor<1x2048x2048xf32> loc(#loc6)
232             top.return %27 : tensor<1x2048x2048xf32> loc(#loc6)
233             } : tensor<1x2048x2048xf32> loc(#loc6)
234             %15 = top.tile {
235                 %18 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
    tensor<1x2048x512xf32, {}> loc(#loc13)
236                 %19 = "top.tileInput"(%9) : (tensor<1x512x1024xf32>) ->
    tensor<1x512x1024xf32, {}> loc(#loc13)
237                 %20 = tensor.empty() : tensor<1x2048x1024xf32, {}> loc(#loc13)
238                 %c0 = arith.constant 0 : index loc(#loc13)
239                 %c0_0 = arith.constant 0 : index loc(#loc13)
240                 %c0_1 = arith.constant 0 : index loc(#loc13)
241                 %c1 = arith.constant 1 : index loc(#loc13)
242                 %c2048 = arith.constant 2048 : index loc(#loc13)
243                 %c1024 = arith.constant 1024 : index loc(#loc13)
244                 %c1_2 = arith.constant 1 : index loc(#loc13)
245                 %c1024_3 = arith.constant 1024 : index loc(#loc13)
246                 %c1024_4 = arith.constant 1024 : index loc(#loc13)
247                 %21 = scf.for %arg1 = %c0 to %c1 step %c1_2 iter_args(%arg2 = %20) ->
    (tensor<1x2048x1024xf32, {}>) {
248                     %23 = scf.for %arg3 = %c0_0 to %c2048 step %c1024_3 iter_args(%arg4
    = %arg2) -> (tensor<1x2048x1024xf32, {}>) {

```

```

249         %24 = scf.for %arg5 = %c0_1 to %c1024 step %c1024_4
iter_args(%arg6 = %arg4) -> (tensor<1x2048x1024xf32, {}>) {
250         %25 = "top.tileInput"(%6) : (tensor<1x2048x512xf32>) ->
tensor<1x2048x512xf32, {}> loc(#loc13)
251         %extracted_slice = tensor.extract_slice %18[%arg1, %arg3, 0] [1,
1024, 512] [1, 1, 1] : tensor<1x2048x512xf32, {}> to tensor<1x1024x512xf32,
{}> loc(#loc13)
252         %26 = "top.tileInput"(%9) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}> loc(#loc13)
253         %extracted_slice_5 = tensor.extract_slice %19[%arg1, 0, %arg5]
[1, 512, 1024] [1, 1, 1] : tensor<1x512x1024xf32, {}> to
tensor<1x512x1024xf32, {}> loc(#loc13)
254         %extracted_slice_6 = tensor.extract_slice %arg6[%arg1, %arg3,
%arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x2048x1024xf32, {}> to
tensor<1x1024x1024xf32, {}> loc(#loc13)
255         %27 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, 1024,
1024) step (1, 32, 512) shared_outs(%arg10 = %extracted_slice_6) ->
(tensor<1x1024x1024xf32, {}>) {
256         %extracted_slice_7 = tensor.extract_slice
%extracted_slice[%arg7, %arg8, 0] [1, 32, 512] [1, 1, 1] :
tensor<1x1024x512xf32, {}> to tensor<1x32x512xf32, {}> loc(#loc13)
257         %extracted_slice_8 = tensor.extract_slice
%extracted_slice_5[%arg7, 0, %arg9] [1, 512, 512] [1, 1, 1] :
tensor<1x512x1024xf32, {}> to tensor<1x512x512xf32, {}> loc(#loc13)
258         %extracted_slice_9 = tensor.extract_slice %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x1024x1024xf32, {}> to
tensor<1x32x512xf32, {}> loc(#loc13)
259         %c0_10 = arith.constant 0 : index loc(#loc13)
260         %c0_11 = arith.constant 0 : index loc(#loc13)
261         %c0_12 = arith.constant 0 : index loc(#loc13)
262         %c1_13 = arith.constant 1 : index loc(#loc13)
263         %c32 = arith.constant 32 : index loc(#loc13)
264         %c512 = arith.constant 512 : index loc(#loc13)
265         %c1_14 = arith.constant 1 : index loc(#loc13)
266         %c32_15 = arith.constant 32 : index loc(#loc13)
267         %c32_16 = arith.constant 32 : index loc(#loc13)
268         %28 = scf.for %arg11 = %c0_10 to %c1_13 step %c1_14
iter_args(%arg12 = %extracted_slice_9) -> (tensor<1x32x512xf32, {}>) {
269         %29 = scf.for %arg13 = %c0_11 to %c32 step %c32_15
iter_args(%arg14 = %arg12) -> (tensor<1x32x512xf32, {}>) {
270         %30 = scf.for %arg15 = %c0_12 to %c512 step %c32_16
iter_args(%arg16 = %arg14) -> (tensor<1x32x512xf32, {}>) {
271         %extracted_slice_17 = tensor.extract_slice
%extracted_slice_7[%arg11, %arg13, 0] [1, 32, 512] [1, 1, 1] :
tensor<1x32x512xf32, {}> to tensor<1x32x512xf32, {}> loc(#loc13)
272         %extracted_slice_18 = tensor.extract_slice
%extracted_slice_8[%arg11, 0, %arg15] [1, 512, 32] [1, 1, 1] :

```

```

tensor<1x512x512xf32, {}> to tensor<1x512x32xf32, {}> loc(#loc13)
273         %extracted_slice_19 = tensor.extract_slice
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x512xf32,
{}> to tensor<1x32x32xf32, {}> loc(#loc13)
274         %31 = linalg.batch_matmul {is_root = true}
ins(%extracted_slice_17, %extracted_slice_18 : tensor<1x32x512xf32, {}>,
tensor<1x512x32xf32, {}>) outs(%extracted_slice_19 : tensor<1x32x32xf32, {}>) -
> tensor<1x32x32xf32, {}> loc(#loc13)
275         %inserted_slice_20 = tensor.insert_slice %31 into
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x32xf32, {}>
into tensor<1x32x512xf32, {}> loc(#loc13)
276         scf.yield %inserted_slice_20 : tensor<1x32x512xf32, {}>
loc(#loc13)
277         } {tiling_level = "L1_tiling"} loc(#loc13)
278         scf.yield %30 : tensor<1x32x512xf32, {}> loc(#loc13)
279         } {tiling_level = "L1_tiling"} loc(#loc13)
280         scf.yield %29 : tensor<1x32x512xf32, {}> loc(#loc13)
281         } {tiling_level = "L1_tiling"} loc(#loc13)
282         scf.forall.in_parallel {
283             tensor.parallel_insert_slice %28 into %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x32x512xf32, {}> into
tensor<1x1024x1024xf32, {}> loc(#loc13)
284             } loc(#loc13)
285             } {tiling_level = "core_sharding"} loc(#loc13)
286             %inserted_slice = tensor.insert_slice %27 into %arg6[%arg1,
%arg3, %arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x1024x1024xf32, {}> into
tensor<1x2048x1024xf32, {}> loc(#loc13)
287             scf.yield %inserted_slice : tensor<1x2048x1024xf32, {}>
loc(#loc13)
288             } {tiling_level = "L2_tiling"} loc(#loc13)
289             scf.yield %24 : tensor<1x2048x1024xf32, {}> loc(#loc13)
290             } {tiling_level = "L2_tiling"} loc(#loc13)
291             scf.yield %23 : tensor<1x2048x1024xf32, {}> loc(#loc13)
292             } {tiling_level = "L2_tiling"} loc(#loc13)
293             %22 = "top.tileOutput"(%21) : (tensor<1x2048x1024xf32, {}>) ->
tensor<1x2048x1024xf32> loc(#loc13)
294             top.return %22 : tensor<1x2048x1024xf32> loc(#loc13)
295             } : tensor<1x2048x1024xf32> loc(#loc13)
296             %16 = top.tile {
297                 %18 = "top.tileInput"(%15) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {}> loc(#loc14)
298                 %19 = "top.tileInput"(%14) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {}> loc(#loc14)
299                 %20 = tensor.empty() : tensor<1x2048x1024xf32, {}> loc(#loc14)
300                 %c0 = arith.constant 0 : index loc(#loc14)
301                 %c0_0 = arith.constant 0 : index loc(#loc14)
302                 %c0_1 = arith.constant 0 : index loc(#loc14)

```

```

303         %c1 = arith.constant 1 : index loc(#loc14)
304         %c2048 = arith.constant 2048 : index loc(#loc14)
305         %c1024 = arith.constant 1024 : index loc(#loc14)
306         %c1_2 = arith.constant 1 : index loc(#loc14)
307         %c1024_3 = arith.constant 1024 : index loc(#loc14)
308         %c1024_4 = arith.constant 1024 : index loc(#loc14)
309         %21 = scf.for %arg1 = %c0 to %c1 step %c1_2 iter_args(%arg2 = %20) ->
(tensor<1x2048x1024xf32, {}>) {
310             %23 = scf.for %arg3 = %c0_0 to %c2048 step %c1024_3 iter_args(%arg4
= %arg2) -> (tensor<1x2048x1024xf32, {}>) {
311                 %24 = scf.for %arg5 = %c0_1 to %c1024 step %c1024_4
iter_args(%arg6 = %arg4) -> (tensor<1x2048x1024xf32, {}>) {
312                     %25 = "top.tileInput"(%14) : (tensor<1x2048x2048xf32>) ->
tensor<1x2048x2048xf32, {}> loc(#loc14)
313                     %extracted_slice = tensor.extract_slice %19[%arg1, %arg3, 0] [1,
1024, 2048] [1, 1, 1] : tensor<1x2048x2048xf32, {}> to tensor<1x1024x2048xf32,
{}> loc(#loc14)
314                     %26 = "top.tileInput"(%15) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32, {}> loc(#loc14)
315                     %extracted_slice_5 = tensor.extract_slice %18[%arg1, 0, %arg5]
[1, 2048, 1024] [1, 1, 1] : tensor<1x2048x1024xf32, {}> to
tensor<1x2048x1024xf32, {}> loc(#loc14)
316                     %extracted_slice_6 = tensor.extract_slice %arg6[%arg1, %arg3,
%arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x2048x1024xf32, {}> to
tensor<1x1024x1024xf32, {}> loc(#loc14)
317                     %27 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, 1024,
1024) step (1, 32, 512) shared_outs(%arg10 = %extracted_slice_6) ->
(tensor<1x1024x1024xf32, {}>) {
318                         %extracted_slice_7 = tensor.extract_slice
%extracted_slice[%arg7, %arg8, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x1024x2048xf32, {}> to tensor<1x32x2048xf32, {}> loc(#loc14)
319                         %extracted_slice_8 = tensor.extract_slice
%extracted_slice_5[%arg7, 0, %arg9] [1, 2048, 512] [1, 1, 1] :
tensor<1x2048x1024xf32, {}> to tensor<1x2048x512xf32, {}> loc(#loc14)
320                         %extracted_slice_9 = tensor.extract_slice %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x1024x1024xf32, {}> to
tensor<1x32x512xf32, {}> loc(#loc14)
321                         %c0_10 = arith.constant 0 : index loc(#loc14)
322                         %c0_11 = arith.constant 0 : index loc(#loc14)
323                         %c0_12 = arith.constant 0 : index loc(#loc14)
324                         %c1_13 = arith.constant 1 : index loc(#loc14)
325                         %c32 = arith.constant 32 : index loc(#loc14)
326                         %c512 = arith.constant 512 : index loc(#loc14)
327                         %c1_14 = arith.constant 1 : index loc(#loc14)
328                         %c32_15 = arith.constant 32 : index loc(#loc14)
329                         %c32_16 = arith.constant 32 : index loc(#loc14)

```

```

330         %28 = scf.for %arg11 = %c0_10 to %c1_13 step %c1_14
iter_args(%arg12 = %extracted_slice_9) -> (tensor<1x32x512xf32, {}>) {
331         %29 = scf.for %arg13 = %c0_11 to %c32 step %c32_15
iter_args(%arg14 = %arg12) -> (tensor<1x32x512xf32, {}>) {
332         %30 = scf.for %arg15 = %c0_12 to %c512 step %c32_16
iter_args(%arg16 = %arg14) -> (tensor<1x32x512xf32, {}>) {
333         %extracted_slice_17 = tensor.extract_slice
%extracted_slice_7[%arg11, %arg13, 0] [1, 32, 2048] [1, 1, 1] :
tensor<1x32x2048xf32, {}> to tensor<1x32x2048xf32, {}> loc(#loc14)
334         %extracted_slice_18 = tensor.extract_slice
%extracted_slice_8[%arg11, 0, %arg15] [1, 2048, 32] [1, 1, 1] :
tensor<1x2048x512xf32, {}> to tensor<1x2048x32xf32, {}> loc(#loc14)
335         %extracted_slice_19 = tensor.extract_slice
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x512xf32,
{}> to tensor<1x32x32xf32, {}> loc(#loc14)
336         %31 = linalg.batch_matmul {is_root = true}
ins(%extracted_slice_17, %extracted_slice_18 : tensor<1x32x2048xf32, {}>,
tensor<1x2048x32xf32, {}>) outs(%extracted_slice_19 : tensor<1x32x32xf32, {}>)
-> tensor<1x32x32xf32, {}> loc(#loc14)
337         %inserted_slice_20 = tensor.insert_slice %31 into
%arg16[%arg11, %arg13, %arg15] [1, 32, 32] [1, 1, 1] : tensor<1x32x32xf32, {}>
into tensor<1x32x512xf32, {}> loc(#loc14)
338         scf.yield %inserted_slice_20 : tensor<1x32x512xf32, {}>
loc(#loc14)
339         } {tiling_level = "L1_tiling"} loc(#loc14)
340         scf.yield %30 : tensor<1x32x512xf32, {}> loc(#loc14)
341         } {tiling_level = "L1_tiling"} loc(#loc14)
342         scf.yield %29 : tensor<1x32x512xf32, {}> loc(#loc14)
343         } {tiling_level = "L1_tiling"} loc(#loc14)
344         scf.forall.in_parallel {
345         tensor.parallel_insert_slice %28 into %arg10[%arg7, %arg8,
%arg9] [1, 32, 512] [1, 1, 1] : tensor<1x32x512xf32, {}> into
tensor<1x1024x1024xf32, {}> loc(#loc14)
346         } loc(#loc14)
347         } {tiling_level = "core_sharding"} loc(#loc14)
348         %inserted_slice = tensor.insert_slice %27 into %arg6[%arg1,
%arg3, %arg5] [1, 1024, 1024] [1, 1, 1] : tensor<1x1024x1024xf32, {}> into
tensor<1x2048x1024xf32, {}> loc(#loc14)
349         scf.yield %inserted_slice : tensor<1x2048x1024xf32, {}>
loc(#loc14)
350         } {tiling_level = "L2_tiling"} loc(#loc14)
351         scf.yield %24 : tensor<1x2048x1024xf32, {}> loc(#loc14)
352         } {tiling_level = "L2_tiling"} loc(#loc14)
353         scf.yield %23 : tensor<1x2048x1024xf32, {}> loc(#loc14)
354         } {tiling_level = "L2_tiling"} loc(#loc14)
355         %22 = "top.tileOutput"(%21) : (tensor<1x2048x1024xf32, {}>) ->
tensor<1x2048x1024xf32> loc(#loc14)

```

```

356         top.return %22 : tensor<1x2048x1024xf32> loc(#loc14)
357     } : tensor<1x2048x1024xf32> loc(#loc14)
358     %17 = "top.fusionOutput"(%16) : (tensor<1x2048x1024xf32>) ->
tensor<1x2048x1024xf32> loc(#loc6)
359     top.return %17 : tensor<1x2048x1024xf32> loc(#loc6)
360 } : tensor<1x2048x1024xf32> loc(#loc6)
361 return %5 : tensor<1x2048x1024xf32> loc(#loc15)
362 } loc(#loc)
363 } loc(#loc)
364

```

非规则形状

代码块

```

1  #map = affine_map<(d0) -> (-d0 + 2004, 1024)>
2  #map1 = affine_map<(d0)[s0] -> (-d0 + s0, 256)>
3  #map2 = affine_map<(d0)[s0] -> (-d0 + s0, 64)>
4  #map3 = affine_map<(d0) -> (-d0 + 2004, 2048)>
5  #map4 = affine_map<(d0)[s0] -> (-d0 + s0, 2048)>
6  module attributes {top.core_num = 4 : i64, top.version = "1.0.0"} {
7      func.func @AttentionGraphFixed(%arg0: tensor<1x2004x512xf32>) ->
tensor<1x2004x1024xf32> {
8          %c128 = arith.constant 128 : index
9          %c2048 = arith.constant 2048 : index
10         %c512 = arith.constant 512 : index
11         %c32 = arith.constant 32 : index
12         %c64 = arith.constant 64 : index
13         %c1024 = arith.constant 1024 : index
14         %c2004 = arith.constant 2004 : index
15         %c1 = arith.constant 1 : index
16         %c0 = arith.constant 0 : index
17         %0 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<1x2004x2004xf32>}> : () -> tensor<1x2004x2004xf32>
18         %1 = "top.constant"() <{value = dense<3.125000e-02> : tensor<f32>}> : () -
> tensor<1x1x1xf32>
19         %2 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32>
20         %3 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32>
21         %4 = "top.constant"() <{value = dense_resource<__elided__> :
tensor<512x1024xf32>}> : () -> tensor<1x512x1024xf32>
22         %5 = top.fusion {
23             %6 = "top.fusionInput"(%arg0) : (tensor<1x2004x512xf32>) ->
tensor<1x2004x512xf32>

```

```

24      %7 = "top.fusionInput"(%0) : (tensor<1x2004x2004xf32>) ->
      tensor<1x2004x2004xf32>
25      %8 = "top.fusionInput"(%1) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32>
26      %9 = "top.fusionInput"(%2) : (tensor<1x512x1024xf32>) ->
      tensor<1x512x1024xf32>
27      %10 = "top.fusionInput"(%3) : (tensor<1x512x1024xf32>) ->
      tensor<1x512x1024xf32>
28      %11 = "top.fusionInput"(%4) : (tensor<1x512x1024xf32>) ->
      tensor<1x512x1024xf32>
29      %12 = top.tile {
30      %18 = "top.tileInput"(%6) : (tensor<1x2004x512xf32>) ->
      tensor<1x2004x512xf32, {}>
31      %19 = "top.tileInput"(%11) : (tensor<1x512x1024xf32>) ->
      tensor<1x512x1024xf32, {}>
32      %20 = tensor.empty() : tensor<1x2004x1024xf32, {}>
33      %21 = scf.for %arg1 = %c0 to %c1 step %c1 iter_args(%arg2 = %20) ->
      (tensor<1x2004x1024xf32, {}>) {
34      %23 = scf.for %arg3 = %c0 to %c2004 step %c1024 iter_args(%arg4 =
      %arg2) -> (tensor<1x2004x1024xf32, {}>) {
35      %24 = scf.for %arg5 = %c0 to %c1024 step %c1024 iter_args(%arg6 =
      %arg4) -> (tensor<1x2004x1024xf32, {}>) {
36      %25 = affine.min #map(%arg3)
37      %extracted_slice = tensor.extract_slice %18[%arg1, %arg3, 0] [1,
      %25, 512] [1, 1, 1] : tensor<1x2004x512xf32, {}> to tensor<1x?x512xf32, {}>
38      %extracted_slice_0 = tensor.extract_slice %19[%arg1, 0, %arg5]
      [1, 512, 1024] [1, 1, 1] : tensor<1x512x1024xf32, {}> to
      tensor<1x512x1024xf32, {}>
39      %extracted_slice_1 = tensor.extract_slice %arg6[%arg1, %arg3,
      %arg5] [1, %25, 1024] [1, 1, 1] : tensor<1x2004x1024xf32, {}> to tensor<1x?
      x1024xf32, {}>
40      %26 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, %25,
      1024) step (1, 256, 1024) shared_outs(%arg10 = %extracted_slice_1) ->
      (tensor<1x?x1024xf32, {}>) {
41      %27 = affine.min #map1(%arg8) [%25]
42      %extracted_slice_2 = tensor.extract_slice %extracted_slice[0,
      %arg8, 0] [1, %27, 512] [1, 1, 1] : tensor<1x?x512xf32, {}> to tensor<1x?
      x512xf32, {}>
43      %extracted_slice_3 = tensor.extract_slice %arg10[0, %arg8, 0]
      [1, %27, 1024] [1, 1, 1] : tensor<1x?x1024xf32, {}> to tensor<1x?x1024xf32, {}>
44      %28 = scf.for %arg11 = %c0 to %c1 step %c1 iter_args(%arg12 =
      %extracted_slice_3) -> (tensor<1x?x1024xf32, {}>) {
45      %29 = scf.for %arg13 = %c0 to %27 step %c64 iter_args(%arg14
      = %arg12) -> (tensor<1x?x1024xf32, {}>) {
46      %30 = scf.for %arg15 = %c0 to %c1024 step %c32
      iter_args(%arg16 = %arg14) -> (tensor<1x?x1024xf32, {}>) {
47      %31 = affine.min #map2(%arg13) [%27]

```

```

48         %extracted_slice_4 = tensor.extract_slice
%extracted_slice_2[%arg11, %arg13, 0] [1, %31, 512] [1, 1, 1] : tensor<1x?
x512xf32, {}> to tensor<1x?x512xf32, {}>
49         %extracted_slice_5 = tensor.extract_slice
%extracted_slice_0[%arg11, 0, %arg15] [1, 512, 32] [1, 1, 1] :
tensor<1x512x1024xf32, {}> to tensor<1x512x32xf32, {}>
50         %extracted_slice_6 = tensor.extract_slice %arg16[%arg11,
%arg13, %arg15] [1, %31, 32] [1, 1, 1] : tensor<1x?x1024xf32, {}> to tensor<1x?
x32xf32, {}>
51         %32 = linalg.batch_matmul {is_root = true}
ins(%extracted_slice_4, %extracted_slice_5 : tensor<1x?x512xf32, {}>,
tensor<1x512x32xf32, {}>) outs(%extracted_slice_6 : tensor<1x?x32xf32, {}>) ->
tensor<1x?x32xf32, {}>
52         %inserted_slice_7 = tensor.insert_slice %32 into
%arg16[%arg11, %arg13, %arg15] [1, %31, 32] [1, 1, 1] : tensor<1x?x32xf32, {}>
into tensor<1x?x1024xf32, {}>
53         scf.yield %inserted_slice_7 : tensor<1x?x1024xf32, {}>
54         } {tiling_level = "L1_tiling"}
55         scf.yield %30 : tensor<1x?x1024xf32, {}>
56         } {tiling_level = "L1_tiling"}
57         scf.yield %29 : tensor<1x?x1024xf32, {}>
58         } {tiling_level = "L1_tiling"}
59         scf.forall.in_parallel {
60             tensor.parallel_insert_slice %28 into %arg10[0, %arg8, 0]
[1, %27, 1024] [1, 1, 1] : tensor<1x?x1024xf32, {}> into tensor<1x?x1024xf32,
{}>
61         }
62         } {tiling_level = "core_sharding"}
63         %inserted_slice = tensor.insert_slice %26 into %arg6[%arg1,
%arg3, %arg5] [1, %25, 1024] [1, 1, 1] : tensor<1x?x1024xf32, {}> into
tensor<1x2004x1024xf32, {}>
64         scf.yield %inserted_slice : tensor<1x2004x1024xf32, {}>
65         } {tiling_level = "L2_tiling"}
66         scf.yield %24 : tensor<1x2004x1024xf32, {}>
67         } {tiling_level = "L2_tiling"}
68         scf.yield %23 : tensor<1x2004x1024xf32, {}>
69         } {tiling_level = "L2_tiling"}
70         %22 = "top.tileOutput"(%21) : (tensor<1x2004x1024xf32, {}>) ->
tensor<1x2004x1024xf32>
71         top.return %22 : tensor<1x2004x1024xf32>
72         } : tensor<1x2004x1024xf32>
73         %13 = top.tile {
74             %18 = "top.tileInput"(%6) : (tensor<1x2004x512xf32>) ->
tensor<1x2004x512xf32, {}>
75             %19 = "top.tileInput"(%10) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}>
76             %20 = tensor.empty() : tensor<1x2004x1024xf32>

```

```

77         %21 = tensor.empty() : tensor<1x1024x2004xf32, {}>
78         %22 = scf.for %arg1 = %c0 to %c1 step %c1 iter_args(%arg2 = %21) ->
(tensor<1x1024x2004xf32, {}>) {
79             %24 = scf.for %arg3 = %c0 to %c1024 step %c512 iter_args(%arg4 =
%arg2) -> (tensor<1x1024x2004xf32, {}>) {
80                 %25 = scf.for %arg5 = %c0 to %c2004 step %c2048 iter_args(%arg6 =
%arg4) -> (tensor<1x1024x2004xf32, {}>) {
81                     %26 = affine.min #map3(%arg5)
82                     %extracted_slice = tensor.extract_slice %18[%arg1, %arg5, 0] [1,
%26, 512] [1, 1, 1] : tensor<1x2004x512xf32, {}> to tensor<1x?x512xf32, {}>
83                     %extracted_slice_0 = tensor.extract_slice %19[%arg1, 0, %arg3]
[1, 512, 512] [1, 1, 1] : tensor<1x512x1024xf32, {}> to tensor<1x512x512xf32,
{}>
84                     %extracted_slice_1 = tensor.extract_slice %20[%arg1, %arg5,
%arg3] [1, %26, 512] [1, 1, 1] : tensor<1x2004x1024xf32> to tensor<1x?x512xf32>
85                     %extracted_slice_2 = tensor.extract_slice %arg6[%arg1, %arg3,
%arg5] [1, 512, %26] [1, 1, 1] : tensor<1x1024x2004xf32, {}> to tensor<1x512x?
xf32, {}>
86                     %27 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, 512,
%26) step (1, 128, 2048) shared_outs(%arg10 = %extracted_slice_2) ->
(tensor<1x512x?xf32, {}>) {
87                         %28 = affine.min #map4(%arg9)[%26]
88                         %extracted_slice_3 = tensor.extract_slice %extracted_slice[0,
%arg9, 0] [1, %28, 512] [1, 1, 1] : tensor<1x?x512xf32, {}> to tensor<1x?
x512xf32, {}>
89                         %extracted_slice_4 = tensor.extract_slice
%extracted_slice_0[0, 0, %arg8] [1, 512, 128] [1, 1, 1] :
tensor<1x512x512xf32, {}> to tensor<1x512x128xf32, {}>
90                         %extracted_slice_5 = tensor.extract_slice
%extracted_slice_1[0, %arg9, %arg8] [1, %28, 128] [1, 1, 1] : tensor<1x?
x512xf32> to tensor<1x?x128xf32>
91                         %extracted_slice_6 = tensor.extract_slice %arg10[0, %arg8,
%arg9] [1, 128, %28] [1, 1, 1] : tensor<1x512x?xf32, {}> to tensor<1x128x?
xf32, {}>
92                         %29 = scf.for %arg11 = %c0 to %c1 step %c1 iter_args(%arg12 =
%extracted_slice_6) -> (tensor<1x128x?xf32, {}>) {
93                             %30 = scf.for %arg13 = %c0 to %c128 step %c32
iter_args(%arg14 = %arg12) -> (tensor<1x128x?xf32, {}>) {
94                                 %31 = scf.for %arg15 = %c0 to %28 step %c64
iter_args(%arg16 = %arg14) -> (tensor<1x128x?xf32, {}>) {
95                                     %32 = affine.min #map2(%arg15)[%28]
96                                     %extracted_slice_7 = tensor.extract_slice
%extracted_slice_3[%arg11, %arg15, 0] [1, %32, 512] [1, 1, 1] : tensor<1x?
x512xf32, {}> to tensor<1x?x512xf32, {}>
97                                     %extracted_slice_8 = tensor.extract_slice
%extracted_slice_4[%arg11, 0, %arg13] [1, 512, 32] [1, 1, 1] :
tensor<1x512x128xf32, {}> to tensor<1x512x32xf32, {}>

```

```

98         %extracted_slice_9 = tensor.extract_slice
%extracted_slice_5[%arg11, %arg15, %arg13] [1, %32, 32] [1, 1, 1] : tensor<1x?
x128xf32> to tensor<1x?x32xf32>
99         %33 = linalg.batch_matmul ins(%extracted_slice_7,
%extracted_slice_8 : tensor<1x?x512xf32, {}>, tensor<1x512x32xf32, {}>)
outs(%extracted_slice_9 : tensor<1x?x32xf32>) -> tensor<1x?x32xf32>
100        %extracted_slice_10 = tensor.extract_slice
%arg16[%arg11, %arg13, %arg15] [1, 32, %32] [1, 1, 1] : tensor<1x128x?xf32,
{}> to tensor<1x32x?xf32, {}>
101        %transposed = linalg.transpose ins(%33 : tensor<1x?
x32xf32>) outs(%extracted_slice_10 : tensor<1x32x?xf32, {}>) permutation = [0,
2, 1] {is_root = true}
102        %inserted_slice_11 = tensor.insert_slice %transposed
into %arg16[%arg11, %arg13, %arg15] [1, 32, %32] [1, 1, 1] : tensor<1x32x?
xf32, {}> into tensor<1x128x?xf32, {}>
103        scf.yield %inserted_slice_11 : tensor<1x128x?xf32, {}>
104        } {tiling_level = "L1_tiling"}
105        scf.yield %31 : tensor<1x128x?xf32, {}>
106        } {tiling_level = "L1_tiling"}
107        scf.yield %30 : tensor<1x128x?xf32, {}>
108        } {tiling_level = "L1_tiling"}
109        scf.forall.in_parallel {
110            tensor.parallel_insert_slice %29 into %arg10[0, %arg8,
%arg9] [1, 128, %28] [1, 1, 1] : tensor<1x128x?xf32, {}> into tensor<1x512x?
xf32, {}>
111        }
112        } {tiling_level = "core_sharding"}
113        %inserted_slice = tensor.insert_slice %27 into %arg6[%arg1,
%arg3, %arg5] [1, 512, %26] [1, 1, 1] : tensor<1x512x?xf32, {}> into
tensor<1x1024x2004xf32, {}>
114        scf.yield %inserted_slice : tensor<1x1024x2004xf32, {}>
115        } {tiling_level = "L2_tiling"}
116        scf.yield %25 : tensor<1x1024x2004xf32, {}>
117        } {tiling_level = "L2_tiling"}
118        scf.yield %24 : tensor<1x1024x2004xf32, {}>
119        } {tiling_level = "L2_tiling"}
120        %23 = "top.tileOutput"(%22) : (tensor<1x1024x2004xf32, {}>) ->
tensor<1x1024x2004xf32>
121        top.return %23 : tensor<1x1024x2004xf32>
122    } : tensor<1x1024x2004xf32>
123    %14 = top.tile {
124        %18 = "top.tileInput"(%12) : (tensor<1x2004x1024xf32>) ->
tensor<1x2004x1024xf32, {}>
125        %19 = "top.tileInput"(%13) : (tensor<1x1024x2004xf32>) ->
tensor<1x1024x2004xf32, {}>
126        %20 = "top.tileInput"(%7) : (tensor<1x2004x2004xf32>) ->
tensor<1x2004x2004xf32, {}>

```

```

127         %21 = "top.tileInput"(%8) : (tensor<1x1x1xf32>) -> tensor<1x1x1xf32,
{}>
128         %22 = tensor.empty() : tensor<1x2004x2004xf32>
129         %23 = tensor.empty() : tensor<1x2004x2004xf32>
130         %24 = tensor.empty() : tensor<1x2004x2004xf32>
131         %25 = tensor.empty() : tensor<1x2004x2004xf32, {}>
132         %26 = scf.for %arg1 = %c0 to %c1 step %c1 iter_args(%arg2 = %25) ->
(tensor<1x2004x2004xf32, {}>) {
133             %28 = scf.for %arg3 = %c0 to %c2004 step %c1024 iter_args(%arg4 =
%arg2) -> (tensor<1x2004x2004xf32, {}>) {
134                 %29 = affine.min #map(%arg3)
135                 %extracted_slice = tensor.extract_slice %18[%arg1, %arg3, 0] [1,
%29, 1024] [1, 1, 1] : tensor<1x2004x1024xf32, {}> to tensor<1x?x1024xf32, {}>
136                 %extracted_slice_0 = tensor.extract_slice %19[%arg1, 0, 0] [1,
1024, 2004] [1, 1, 1] : tensor<1x1024x2004xf32, {}> to tensor<1x1024x2004xf32,
{}>
137                 %extracted_slice_1 = tensor.extract_slice %22[%arg1, %arg3, 0] [1,
%29, 2004] [1, 1, 1] : tensor<1x2004x2004xf32> to tensor<1x?x2004xf32>
138                 %extracted_slice_2 = tensor.extract_slice %21[%arg1, 0, 0] [1, 1,
1] [1, 1, 1] : tensor<1x1x1xf32, {}> to tensor<1x1x1xf32, {}>
139                 %extracted_slice_3 = tensor.extract_slice %23[%arg1, %arg3, 0] [1,
%29, 2004] [1, 1, 1] : tensor<1x2004x2004xf32> to tensor<1x?x2004xf32>
140                 %extracted_slice_4 = tensor.extract_slice %20[%arg1, %arg3, 0] [1,
%29, 2004] [1, 1, 1] : tensor<1x2004x2004xf32, {}> to tensor<1x?x2004xf32, {}>
141                 %extracted_slice_5 = tensor.extract_slice %24[%arg1, %arg3, 0] [1,
%29, 2004] [1, 1, 1] : tensor<1x2004x2004xf32> to tensor<1x?x2004xf32>
142                 %extracted_slice_6 = tensor.extract_slice %arg4[%arg1, %arg3, 0]
[1, %29, 2004] [1, 1, 1] : tensor<1x2004x2004xf32, {}> to tensor<1x?x2004xf32,
{}>
143                 %30 = scf.forall (%arg5, %arg6) = (0, 0) to (1, %29) step (1, 256)
shared_outs(%arg7 = %extracted_slice_6) -> (tensor<1x?x2004xf32, {}>) {
144                     %31 = affine.min #map1(%arg6) [%29]
145                     %extracted_slice_7 = tensor.extract_slice %extracted_slice[0,
%arg6, 0] [1, %31, 1024] [1, 1, 1] : tensor<1x?x1024xf32, {}> to tensor<1x?
x1024xf32, {}>
146                     %extracted_slice_8 = tensor.extract_slice %extracted_slice_1[0,
%arg6, 0] [1, %31, 2004] [1, 1, 1] : tensor<1x?x2004xf32> to tensor<1x?
x2004xf32>
147                     %extracted_slice_9 = tensor.extract_slice %extracted_slice_3[0,
%arg6, 0] [1, %31, 2004] [1, 1, 1] : tensor<1x?x2004xf32> to tensor<1x?
x2004xf32>
148                     %extracted_slice_10 = tensor.extract_slice %extracted_slice_4[0,
%arg6, 0] [1, %31, 2004] [1, 1, 1] : tensor<1x?x2004xf32, {}> to tensor<1x?
x2004xf32, {}>
149                     %extracted_slice_11 = tensor.extract_slice %extracted_slice_5[0,
%arg6, 0] [1, %31, 2004] [1, 1, 1] : tensor<1x?x2004xf32> to tensor<1x?
x2004xf32>

```

```

150         %extracted_slice_12 = tensor.extract_slice %arg7[0, %arg6, 0]
[1, %31, 2004] [1, 1, 1] : tensor<1x?x2004xf32, {}> to tensor<1x?x2004xf32, {}>
151         %32 = scf.for %arg8 = %c0 to %c1 step %c1 iter_args(%arg9 =
%extracted_slice_12) -> (tensor<1x?x2004xf32, {}>) {
152             %33 = scf.for %arg10 = %c0 to %31 step %c64 iter_args(%arg11 =
%arg9) -> (tensor<1x?x2004xf32, {}>) {
153                 %34 = affine.min #map2(%arg10)[%31]
154                 %extracted_slice_13 = tensor.extract_slice
%extracted_slice_7[%arg8, %arg10, 0] [1, %34, 1024] [1, 1, 1] : tensor<1x?
x1024xf32, {}> to tensor<1x?x1024xf32, {}>
155                 %extracted_slice_14 = tensor.extract_slice
%extracted_slice_0[%arg8, 0, 0] [1, 1024, 2004] [1, 1, 1] :
tensor<1x1024x2004xf32, {}> to tensor<1x1024x2004xf32, {}>
156                 %extracted_slice_15 = tensor.extract_slice
%extracted_slice_8[%arg8, %arg10, 0] [1, %34, 2004] [1, 1, 1] : tensor<1x?
x2004xf32> to tensor<1x?x2004xf32>
157                 %35 = linalg.batch_matmul ins(%extracted_slice_13,
%extracted_slice_14 : tensor<1x?x1024xf32, {}>, tensor<1x1024x2004xf32, {}>)
outs(%extracted_slice_15 : tensor<1x?x2004xf32>) -> tensor<1x?x2004xf32>
158                 %extracted_slice_16 = tensor.extract_slice
%extracted_slice_2[%arg8, 0, 0] [1, 1, 1] [1, 1, 1] : tensor<1x1x1xf32, {}> to
tensor<1x1x1xf32, {}>
159                 %extracted_slice_17 = tensor.extract_slice
%extracted_slice_9[%arg8, %arg10, 0] [1, %34, 2004] [1, 1, 1] : tensor<1x?
x2004xf32> to tensor<1x?x2004xf32>
160                 %36 = linalg_ext.mul ins(%35, %extracted_slice_16 :
tensor<1x?x2004xf32>, tensor<1x1x1xf32, {}>) outs(%extracted_slice_17 :
tensor<1x?x2004xf32>) -> tensor<1x?x2004xf32>
161                 %extracted_slice_18 = tensor.extract_slice
%extracted_slice_10[%arg8, %arg10, 0] [1, %34, 2004] [1, 1, 1] : tensor<1x?
x2004xf32, {}> to tensor<1x?x2004xf32, {}>
162                 %extracted_slice_19 = tensor.extract_slice
%extracted_slice_11[%arg8, %arg10, 0] [1, %34, 2004] [1, 1, 1] : tensor<1x?
x2004xf32> to tensor<1x?x2004xf32>
163                 %37 = linalg.add ins(%36, %extracted_slice_18 : tensor<1x?
x2004xf32>, tensor<1x?x2004xf32, {}>) outs(%extracted_slice_19 : tensor<1x?
x2004xf32>) -> tensor<1x?x2004xf32>
164                 %extracted_slice_20 = tensor.extract_slice %arg11[%arg8,
%arg10, 0] [1, %34, 2004] [1, 1, 1] : tensor<1x?x2004xf32, {}> to tensor<1x?
x2004xf32, {}>
165                 %38 = linalg.softmax {is_root = true} dimension(2) ins(%37 :
tensor<1x?x2004xf32>) outs(%extracted_slice_20 : tensor<1x?x2004xf32, {}>) ->
tensor<1x?x2004xf32, {}>
166                 %inserted_slice_21 = tensor.insert_slice %38 into
%arg11[%arg8, %arg10, 0] [1, %34, 2004] [1, 1, 1] : tensor<1x?x2004xf32, {}>
into tensor<1x?x2004xf32, {}>
167                 scf.yield %inserted_slice_21 : tensor<1x?x2004xf32, {}>

```

```

168         } {tiling_level = "L1_tiling"}
169         scf.yield %33 : tensor<1x?x2004xf32, {}>
170     } {tiling_level = "L1_tiling"}
171     scf.forall.in_parallel {
172         tensor.parallel_insert_slice %32 into %arg7[0, %arg6, 0] [1,
%31, 2004] [1, 1, 1] : tensor<1x?x2004xf32, {}> into tensor<1x?x2004xf32, {}>
173     }
174     } {tiling_level = "core_sharding"}
175     %inserted_slice = tensor.insert_slice %30 into %arg4[%arg1, %arg3,
0] [1, %29, 2004] [1, 1, 1] : tensor<1x?x2004xf32, {}> into
tensor<1x2004x2004xf32, {}>
176     scf.yield %inserted_slice : tensor<1x2004x2004xf32, {}>
177     } {tiling_level = "L2_tiling"}
178     scf.yield %28 : tensor<1x2004x2004xf32, {}>
179     } {tiling_level = "L2_tiling"}
180     %27 = "top.tileOutput"(%26) : (tensor<1x2004x2004xf32, {}>) ->
tensor<1x2004x2004xf32>
181     top.return %27 : tensor<1x2004x2004xf32>
182     } : tensor<1x2004x2004xf32>
183     %15 = top.tile {
184         %18 = "top.tileInput"(%6) : (tensor<1x2004x512xf32>) ->
tensor<1x2004x512xf32, {}>
185         %19 = "top.tileInput"(%9) : (tensor<1x512x1024xf32>) ->
tensor<1x512x1024xf32, {}>
186         %20 = tensor.empty() : tensor<1x2004x1024xf32, {}>
187         %21 = scf.for %arg1 = %c0 to %c1 step %c1 iter_args(%arg2 = %20) ->
(tensor<1x2004x1024xf32, {}>) {
188             %23 = scf.for %arg3 = %c0 to %c2004 step %c1024 iter_args(%arg4 =
%arg2) -> (tensor<1x2004x1024xf32, {}>) {
189                 %24 = scf.for %arg5 = %c0 to %c1024 step %c1024 iter_args(%arg6 =
%arg4) -> (tensor<1x2004x1024xf32, {}>) {
190                     %25 = affine.min #map(%arg3)
191                     %extracted_slice = tensor.extract_slice %18[%arg1, %arg3, 0] [1,
%25, 512] [1, 1, 1] : tensor<1x2004x512xf32, {}> to tensor<1x?x512xf32, {}>
192                     %extracted_slice_0 = tensor.extract_slice %19[%arg1, 0, %arg5]
[1, 512, 1024] [1, 1, 1] : tensor<1x512x1024xf32, {}> to
tensor<1x512x1024xf32, {}>
193                     %extracted_slice_1 = tensor.extract_slice %arg6[%arg1, %arg3,
%arg5] [1, %25, 1024] [1, 1, 1] : tensor<1x2004x1024xf32, {}> to tensor<1x?
x1024xf32, {}>
194                     %26 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, %25,
1024) step (1, 256, 1024) shared_outs(%arg10 = %extracted_slice_1) ->
(tensor<1x?x1024xf32, {}>) {
195                         %27 = affine.min #map1(%arg8)[%25]
196                         %extracted_slice_2 = tensor.extract_slice %extracted_slice[0,
%arg8, 0] [1, %27, 512] [1, 1, 1] : tensor<1x?x512xf32, {}> to tensor<1x?
x512xf32, {}>

```

```

197         %extracted_slice_3 = tensor.extract_slice %arg10[0, %arg8, 0]
[1, %27, 1024] [1, 1, 1] : tensor<1x?x1024xf32, {}> to tensor<1x?x1024xf32, {}>
198         %28 = scf.for %arg11 = %c0 to %c1 step %c1 iter_args(%arg12 =
%extracted_slice_3) -> (tensor<1x?x1024xf32, {}>) {
199             %29 = scf.for %arg13 = %c0 to %27 step %c64 iter_args(%arg14
= %arg12) -> (tensor<1x?x1024xf32, {}>) {
200                 %30 = scf.for %arg15 = %c0 to %c1024 step %c32
iter_args(%arg16 = %arg14) -> (tensor<1x?x1024xf32, {}>) {
201                     %31 = affine.min #map2(%arg13)[%27]
202                     %extracted_slice_4 = tensor.extract_slice
%extracted_slice_2[%arg11, %arg13, 0] [1, %31, 512] [1, 1, 1] : tensor<1x?
x512xf32, {}> to tensor<1x?x512xf32, {}>
203                     %extracted_slice_5 = tensor.extract_slice
%extracted_slice_0[%arg11, 0, %arg15] [1, 512, 32] [1, 1, 1] :
tensor<1x512x1024xf32, {}> to tensor<1x512x32xf32, {}>
204                     %extracted_slice_6 = tensor.extract_slice %arg16[%arg11,
%arg13, %arg15] [1, %31, 32] [1, 1, 1] : tensor<1x?x1024xf32, {}> to tensor<1x?
x32xf32, {}>
205                     %32 = linalg.batch_matmul {is_root = true}
ins(%extracted_slice_4, %extracted_slice_5 : tensor<1x?x512xf32, {}>,
tensor<1x512x32xf32, {}>) outs(%extracted_slice_6 : tensor<1x?x32xf32, {}>) ->
tensor<1x?x32xf32, {}>
206                     %inserted_slice_7 = tensor.insert_slice %32 into
%arg16[%arg11, %arg13, %arg15] [1, %31, 32] [1, 1, 1] : tensor<1x?x32xf32, {}>
into tensor<1x?x1024xf32, {}>
207                     scf.yield %inserted_slice_7 : tensor<1x?x1024xf32, {}>
208                     } {tiling_level = "L1_tiling"}
209                     scf.yield %30 : tensor<1x?x1024xf32, {}>
210                     } {tiling_level = "L1_tiling"}
211                     scf.yield %29 : tensor<1x?x1024xf32, {}>
212                     } {tiling_level = "L1_tiling"}
213                     scf.forall.in_parallel {
214                         tensor.parallel_insert_slice %28 into %arg10[0, %arg8, 0]
[1, %27, 1024] [1, 1, 1] : tensor<1x?x1024xf32, {}> into tensor<1x?x1024xf32,
{}>
215                     }
216                     } {tiling_level = "core_sharding"}
217                     %inserted_slice = tensor.insert_slice %26 into %arg6[%arg1,
%arg3, %arg5] [1, %25, 1024] [1, 1, 1] : tensor<1x?x1024xf32, {}> into
tensor<1x2004x1024xf32, {}>
218                     scf.yield %inserted_slice : tensor<1x2004x1024xf32, {}>
219                     } {tiling_level = "L2_tiling"}
220                     scf.yield %24 : tensor<1x2004x1024xf32, {}>
221                     } {tiling_level = "L2_tiling"}
222                     scf.yield %23 : tensor<1x2004x1024xf32, {}>
223                     } {tiling_level = "L2_tiling"}

```

```

224         %22 = "top.tileOutput"(%21) : (tensor<1x2004x1024xf32, {}>) ->
tensor<1x2004x1024xf32>
225         top.return %22 : tensor<1x2004x1024xf32>
226     } : tensor<1x2004x1024xf32>
227     %16 = top.tile {
228         %18 = "top.tileInput"(%15) : (tensor<1x2004x1024xf32>) ->
tensor<1x2004x1024xf32, {}>
229         %19 = "top.tileInput"(%14) : (tensor<1x2004x2004xf32>) ->
tensor<1x2004x2004xf32, {}>
230         %20 = tensor.empty() : tensor<1x2004x1024xf32, {}>
231         %21 = scf.for %arg1 = %c0 to %c1 step %c1 iter_args(%arg2 = %20) ->
(tensor<1x2004x1024xf32, {}>) {
232             %23 = scf.for %arg3 = %c0 to %c2004 step %c1024 iter_args(%arg4 =
%arg2) -> (tensor<1x2004x1024xf32, {}>) {
233                 %24 = scf.for %arg5 = %c0 to %c1024 step %c1024 iter_args(%arg6 =
%arg4) -> (tensor<1x2004x1024xf32, {}>) {
234                     %25 = affine.min #map(%arg3)
235                     %extracted_slice = tensor.extract_slice %19[%arg1, %arg3, 0] [1,
%25, 2004] [1, 1, 1] : tensor<1x2004x2004xf32, {}> to tensor<1x?x2004xf32, {}>
236                     %extracted_slice_0 = tensor.extract_slice %18[%arg1, 0, %arg5]
[1, 2004, 1024] [1, 1, 1] : tensor<1x2004x1024xf32, {}> to
tensor<1x2004x1024xf32, {}>
237                     %extracted_slice_1 = tensor.extract_slice %arg6[%arg1, %arg3,
%arg5] [1, %25, 1024] [1, 1, 1] : tensor<1x2004x1024xf32, {}> to tensor<1x?
x1024xf32, {}>
238                     %26 = scf.forall (%arg7, %arg8, %arg9) = (0, 0, 0) to (1, %25,
1024) step (1, 256, 1024) shared_outs(%arg10 = %extracted_slice_1) ->
(tensor<1x?x1024xf32, {}>) {
239                         %27 = affine.min #map1(%arg8)[%25]
240                         %extracted_slice_2 = tensor.extract_slice %extracted_slice[0,
%arg8, 0] [1, %27, 2004] [1, 1, 1] : tensor<1x?x2004xf32, {}> to tensor<1x?
x2004xf32, {}>
241                         %extracted_slice_3 = tensor.extract_slice %arg10[0, %arg8, 0]
[1, %27, 1024] [1, 1, 1] : tensor<1x?x1024xf32, {}> to tensor<1x?x1024xf32, {}>
242                         %28 = scf.for %arg11 = %c0 to %c1 step %c1 iter_args(%arg12 =
%extracted_slice_3) -> (tensor<1x?x1024xf32, {}>) {
243                             %29 = scf.for %arg13 = %c0 to %27 step %c64 iter_args(%arg14
= %arg12) -> (tensor<1x?x1024xf32, {}>) {
244                                 %30 = scf.for %arg15 = %c0 to %c1024 step %c32
iter_args(%arg16 = %arg14) -> (tensor<1x?x1024xf32, {}>) {
245                                     %31 = affine.min #map2(%arg13)[%27]
246                                     %extracted_slice_4 = tensor.extract_slice
%extracted_slice_2[%arg11, %arg13, 0] [1, %31, 2004] [1, 1, 1] : tensor<1x?
x2004xf32, {}> to tensor<1x?x2004xf32, {}>
247                                     %extracted_slice_5 = tensor.extract_slice
%extracted_slice_0[%arg11, 0, %arg15] [1, 2004, 32] [1, 1, 1] :
tensor<1x2004x1024xf32, {}> to tensor<1x2004x32xf32, {}>

```

```

248         %extracted_slice_6 = tensor.extract_slice %arg16[%arg11,
%arg13, %arg15] [1, %31, 32] [1, 1, 1] : tensor<1x?x1024xf32, {}> to tensor<1x?
x32xf32, {}>
249         %32 = linalg.batch_matmul {is_root = true}
ins(%extracted_slice_4, %extracted_slice_5 : tensor<1x?x2004xf32, {}>,
tensor<1x2004x32xf32, {}>) outs(%extracted_slice_6 : tensor<1x?x32xf32, {}>) -
> tensor<1x?x32xf32, {}>
250         %inserted_slice_7 = tensor.insert_slice %32 into
%arg16[%arg11, %arg13, %arg15] [1, %31, 32] [1, 1, 1] : tensor<1x?x32xf32, {}>
into tensor<1x?x1024xf32, {}>
251         scf.yield %inserted_slice_7 : tensor<1x?x1024xf32, {}>
252         } {tiling_level = "L1_tiling"}
253         scf.yield %30 : tensor<1x?x1024xf32, {}>
254         } {tiling_level = "L1_tiling"}
255         scf.yield %29 : tensor<1x?x1024xf32, {}>
256         } {tiling_level = "L1_tiling"}
257         scf.forall.in_parallel {
258             tensor.parallel_insert_slice %28 into %arg10[0, %arg8, 0]
[1, %27, 1024] [1, 1, 1] : tensor<1x?x1024xf32, {}> into tensor<1x?x1024xf32,
{}>
259             }
260             } {tiling_level = "core_sharding"}
261             %inserted_slice = tensor.insert_slice %26 into %arg6[%arg1,
%arg3, %arg5] [1, %25, 1024] [1, 1, 1] : tensor<1x?x1024xf32, {}> into
tensor<1x2004x1024xf32, {}>
262             scf.yield %inserted_slice : tensor<1x2004x1024xf32, {}>
263             } {tiling_level = "L2_tiling"}
264             scf.yield %24 : tensor<1x2004x1024xf32, {}>
265             } {tiling_level = "L2_tiling"}
266             scf.yield %23 : tensor<1x2004x1024xf32, {}>
267             } {tiling_level = "L2_tiling"}
268             %22 = "top.tileOutput"(%21) : (tensor<1x2004x1024xf32, {}>) ->
tensor<1x2004x1024xf32>
269             top.return %22 : tensor<1x2004x1024xf32>
270             } : tensor<1x2004x1024xf32>
271             %17 = "top.fusionOutput"(%16) : (tensor<1x2004x1024xf32>) ->
tensor<1x2004x1024xf32>
272             top.return %17 : tensor<1x2004x1024xf32>
273             } : tensor<1x2004x1024xf32>
274             return %5 : tensor<1x2004x1024xf32>
275         }
276     }
277

```

